

09:00-09:50 · 1일차 1차시

LLM Agent & 업무자동화

답을 잘하는 AI에서, 일을 끝내는 Workflow로

Agent와 문제 정의

09:00-09:12 강의 · 09:12-09:20 강사 시연 · 09:20-09:42 개인 Ideation · 09:42-09:50 확인

지금 배우는 내용	이 차시에서 이해할 것	함께 챙길 내용
Chatbot과 Agent의 차이 · 자동화 범위	모델보다 먼저 입력·결과·금지 행동을 정한다	자동화할 일 한 문장

차성재 · 무신사 Agentic AI Side PM
서울시립대·아주대 AI 겸임교수

2026.08.23 SUN
09:00-18:00 · LUNCH 14:00-15:00

첫날은 점심시간이 14시입니다

강사의 오후 주요 일정으로 이번 주만 1시간 점심시간을 14:00-15:00에 운영합니다.

11:30-12:00 쉬는 시간에는 빠른 점심 또는 간식을 미리 드시길 권합니다.

12-13시에 점심시간을 드리지 못하게 되었습니다. 불편을 드려 죄송합니다. π.π

09:00-12:00	1-3차시	09:00-11:30 수업 (11:30-12:00 쉬는 시간)	빠른 점심·간식 권장
12:00-14:00	4-5차시	12:00-13:40 수업 (13:40-14:00 쉬는 시간)	총 2시간
14:00-15:00	LUNCH	점심시간	이번 주만 14시 시작
15:00-17:30	6-8차시	15:00-17:00 수업 (17:00-17:30 쉬는 시간)	총 2시간 30분
17:30-18:00	Q&A	질문·실행 오류 복구·Exit Ticket	쉬는 시간 뒤 시작

1일차는 여덟 개 차시로 진행합니다

현재 차시의 시간·내용·활동을 확인하고, 쉬는 시간 전에는 실행 상태를 저장합니다.

시간	슬라이드	진행 내용	강의·시연·개인 활동	수강생이 남길 것
09:00-09:50	1-29	1일차 1차시 · Agent와 문제 정의	강의 12 · 시연 8 · Ideation 22 · 확인 8분	자동화할 일 한 문장
09:50-10:40	30-64	1일차 2차시 · Tool Calling과 실행 권한	강의 15 · 시연 10 · 코드 실행 20 · 확인 5분	허용할 도구와 막을 행동
10:40-11:30	65-99	1일차 3차시 · 한국어 데이터와 PBL 사례	강의 14 · 시연 10 · 자료 확인 21 · 저장 5분	입력·결과·근거 예시
11:30-12:00	-	쉬는 시간	1-3차시 종료 후 휴식	빠른 점심 또는 간식 권장
12:00-12:50	100-134	1일차 4차시 · Python·Jupyter·Ollama 환경	강의 8 · 시연 10 · 환경 세팅 24 · 확인 8분	상태 JSON·4 passed
12:50-13:40	135-168	1일차 5차시 · Ollama Tool Calling·안전 실행	강의 10 · 시연 10 · 코드 실행 25 · 확인 5분	정상·차단·fallback·25 passed
13:40-14:00	-	쉬는 시간	4-5차시 종료 후 휴식	점심 전 실행 상태 저장
14:00-15:00	-	점심시간	1시간	14:55까지 복귀
15:00-15:40	169-202	1일차 6차시 · LangChain LCEL·로컬 LLM	강의 8 · 시연 10 · 코드 실행 17 · 확인 5분	typed output·provider 비교
15:40-16:20	203-236	1일차 7차시 · LangChain·LangGraph	강의 8 · 시연 7 · 코드 실행 23 · 확인 2분	LCEL·승인·수정·거절 상태
16:20-17:00	237-269	1일차 8차시 · STT와 LangSmith	강의 8 · 시연 8 · 코드 실행 17 · 판단 7분	품질 기준과 READY/HOLD 판단
17:00-17:30	-	쉬는 시간	6-8차시 종료 후 휴식	질문 정리
17:30-18:00	270	Q&A · 실행 오류 복구 · Exit Ticket	질문 15 · 복구 10 · Exit 5분	오늘의 핵심 세 문장

머신러닝·딥러닝·LLM을 거쳐, 지금은 Agent 제품을 만듭니다

차성재 · 무신사 Agentic AI Side PM · 서울시립대/아주대학교 AI 부문 겸임교수

산업	데이터·AI	풀었던 문제	역할
금융	정형 데이터 AutoML·MLOps	은행·카드·보험사의 예측과 의사결정을 자체 AutoML 솔루션으로 자동화	에이젠글로벌 ML Team Leader
의료	대장내시경 영상 DL·CVOps	실시간 영상에서 용종 탐지·진단을 보조하는 의료 AI 서비스를 제작·운영	아이넥스코퍼레이션 AI Engineer
교육	말하기·글쓰기·상담 LLMOps·AICC	7만 명 영어 평가를 3-5일에서 약 10초로 단축하고 상담 전·중·후 AI를 총괄	크레버스 AI Engineer·AI PM
이커머스	고객·상품 맥락 Agentic AI	AID Chat·쇼핑 탐색·Voice·AI 해설·SEO/GEO 등 고객 접점 Agent 제품을 담당	무신사 Agentic AI Side PM

강의·멘토 · 2개 대학원 AI 겸임교수 / KT·Kakao AX / 연구 AX / 재직자·취준생 AI Native / 출제·심사

먼저 네 단어의 역할만 잡고 갑니다

용어	쉽게 말하면	오늘 쓰는 곳
생성형 AI Generative AI	학습한 패턴을 바탕으로 새로운 텍스트·이미지·코드를 만들어 내는 AI이다.	회의 원문을 읽고 요약 초안을 만든 뒤 사람이 검토한다.
LLM Large Language Model	아주 많은 텍스트로 학습해 다음에 올 말의 가능성을 계산하는 언어 모델이다.	회의에서 결정·담당자·기한 후보를 구조화한다.
Agent Goal + State + Tools + Loop	목표를 받아 현재 상태를 보고, 도구를 선택하고, 결과를 확인하며 다음 행동을 정하는 실행 구조이다.	파일 읽기 → 요약 → 검증 → 승인 → 기록
Harness Engineering Instructions + Tools + Tests	모델 주변에 지시문·도구·권한·테스트·관측 장치를 설계하는 방식이다.	Codex·Claude에 작은 작업을 주고 테스트와 diff로 결과를 확인한다.

재직자와 구직자는 같은 기술을 다르게 증명한다

재직자는 도입 통제, 구직자는 재현 가능한 포트폴리오를 강조한다.

- 재직자: 보안·업무시간·승인
- 구직자: README·테스트·Git history
- 공통: 실제 demo

제품과 데모가 갈리는 지점 | 데모는 '된다'를 보여주고, 제품은 '안 될 때'를 설명한다.

수업은 네 번의 움직임을 반복합니다

먼저 보고, 그대로 따라하고, 일부러 틀려본 뒤, 성공 증거를 남깁니다.

1

강사 화면을 먼저 봅니다

완성 결과와 클릭·명령 순서를 먼저 확인합니다.

2

같은 화면을 따라 만듭니다

한 단계씩 멈추며 내 화면과 비교합니다.

3

일부러 한 번 틀려봅니다

입력값 하나를 바꿔 실패 메시지를 직접 확인합니다.

4

다시 실행해 확인합니다

원상 복구한 뒤 test와 결과 파일을 남깁니다.

막히면 “안 돼요”보다 네 가지를 알려주세요

“안 돼요”

어디서부터 확인해야 할지
알기 어렵습니다

대신 아래 네 가지를 알려주세요

- | | | |
|----|---------|-----------------|
| 01 | 첫 오류 문장 | 화면에 보이는 그대로 |
| 02 | 직전 행동 | 누른 버튼 또는 입력한 명령 |
| 03 | 내 실행 환경 | 운영체제와 Python 버전 |
| 04 | 기대한 결과 | 원래 나와야 했던 화면 |

예시 | macOS · Python 3.11 · pip install 뒤 ModuleNotFoundError · notebook가 열릴 것으로 기대

실습 속도가 달라도, 다음 행동은 분명합니다

빠르거나 느린 것이 아니라, 지금 내 상태에 맞는 행동 하나를 고르면 됩니다.

지금 내 상태	지금 할 일	다음에 남길 것
바로 다시 실행할 수 있어요	혼자 한 번 더 실행하고 결과를 비교합니다.	성공 화면 1장
장표를 따라가는 중이에요	현재 표시된 단계까지만 끝내고 강사 화면과 비교합니다.	현재 단계 체크
오류에서 멈췄어요	채팅창에 첫 오류 한 줄을 남긴 뒤 복구 파일에서 다시 시작합니다.	오류 문장 1줄
먼저 끝냈어요	실패 입력을 하나 더 넣고 왜 막혔는지 기록합니다.	반례와 이유

막혔을 때 조용히 기다리지 않아도 됩니다. 첫 오류 한 줄이 가장 좋은 질문입니다.

실습 완료는 “작동했다”가 아니라 “다시 확인할 수 있다”입니다

다음 네 가지 중 하나라도 없으면, 아직 실습이 끝난 것이 아닙니다.

완료 체크

- ✓ 결과 파일이 저장되어 있습니다
- ✓ 같은 명령을 다시 실행해도 성공합니다
- ✓ test 또는 schema 검사가 통과합니다
- ✓ 왜 안전한지 한 문장으로 설명할 수 있습니다

다시 확인할 수 있는 화면

```
$ python3 -m src.day1_agent  
status: SUCCESS
```

```
$ python3.12 -m pytest -q  
25 passed
```

```
$ git status --short  
M src/day1_agent.py
```

오늘 남길 것 | 실행 화면 1장 + test 결과 1줄 + 내가 설명한 안전 기준 1문장

강사의 완성 흐름은 이 네 화면에서 시작합니다

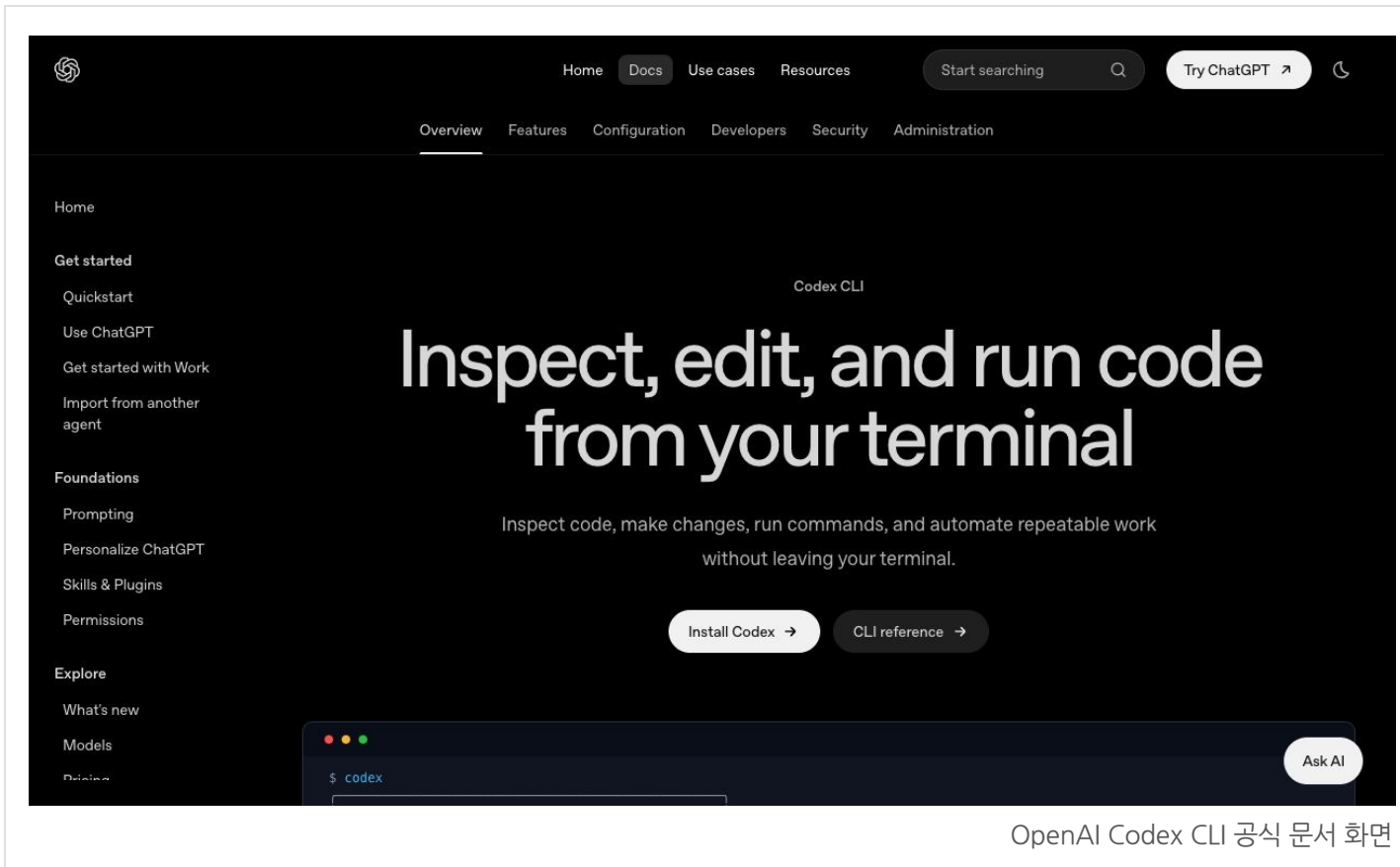
Agent 도구 → 저장소 → 실제 코드 → test 증거로 이어지는 출발 경로

도구·서비스	도메인·역할	이번 강의에서 쓰는 이유	유사 도구와의 차이
VS Code	IDE·주 실습 공간	코드·터미널·Notebook·Git을 한 화면에서 연결	PyCharm보다 가볍고 확장 기반
Jupyter	설명과 셀 실행	처음 배우는 사람이 단계별 결과를 바로 확인	Colab은 설치가 없지만 세션·자원 제약
Git·GitHub	버전·포트폴리오	diff·commit·README로 완료 증거를 남김	Drive 공유보다 변경 이력이 선명
Codex·Claude	Coding Agent·가속	spec→diff→test 데모와 코드 검토를 빠르게 반복	필수 도구가 아닌 선택적 Harness

Codex CLI · 저장소를 읽고 실행하는 Agent

이 화면에서 딱 볼 것

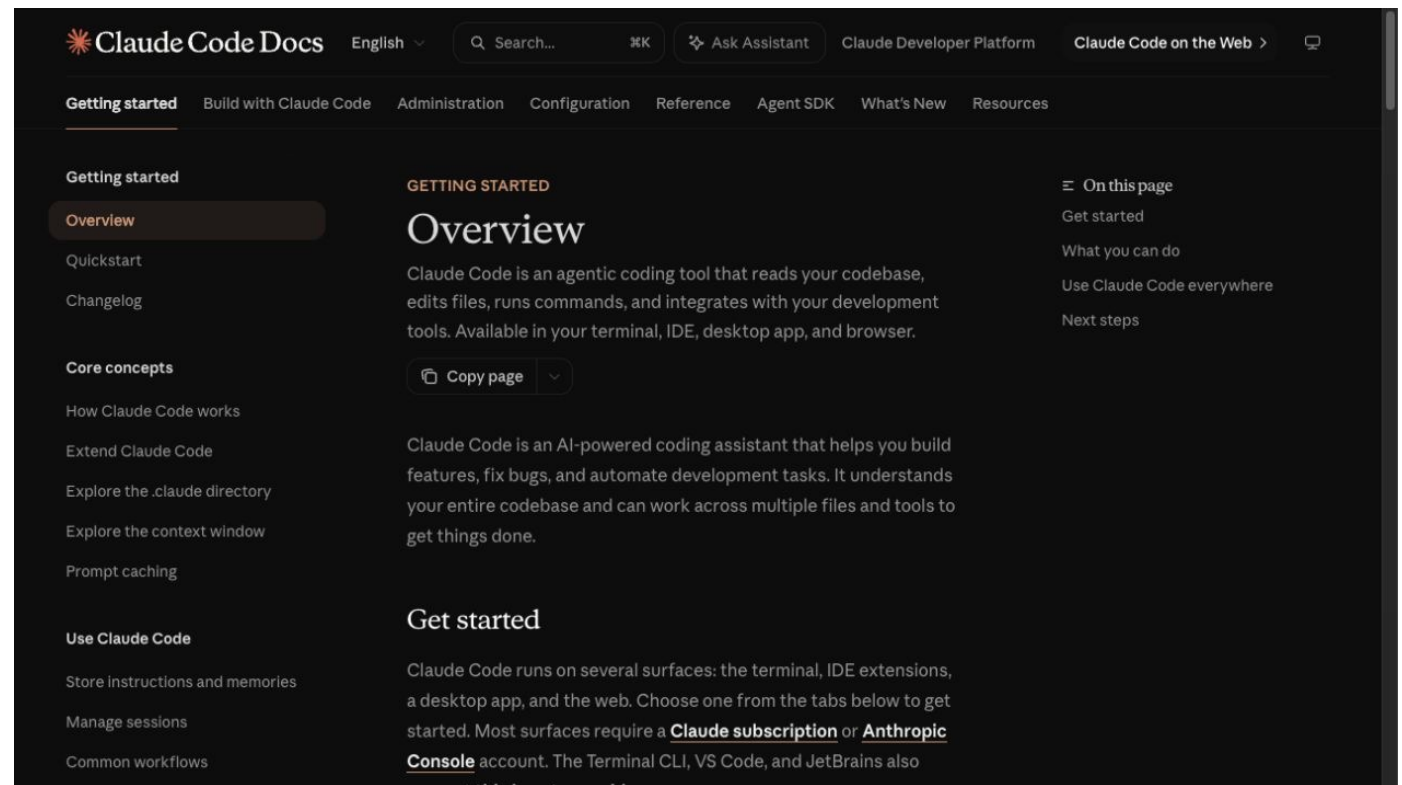
- 작업 디렉터리에서 시작
- 요구사항을 파일·테스트로 구체화
- diff와 명령 결과를 사람이 검토
- 학생 필수 경로는 로컬 코드



Claude Code · 터미널 중심 Harness

이 화면에서 딱 볼 것

- 코드베이스 맥락 읽기
- 작업을 작은 단계로 지시
- 변경·테스트·설명 확인
- 구독·정책은 별도 확인

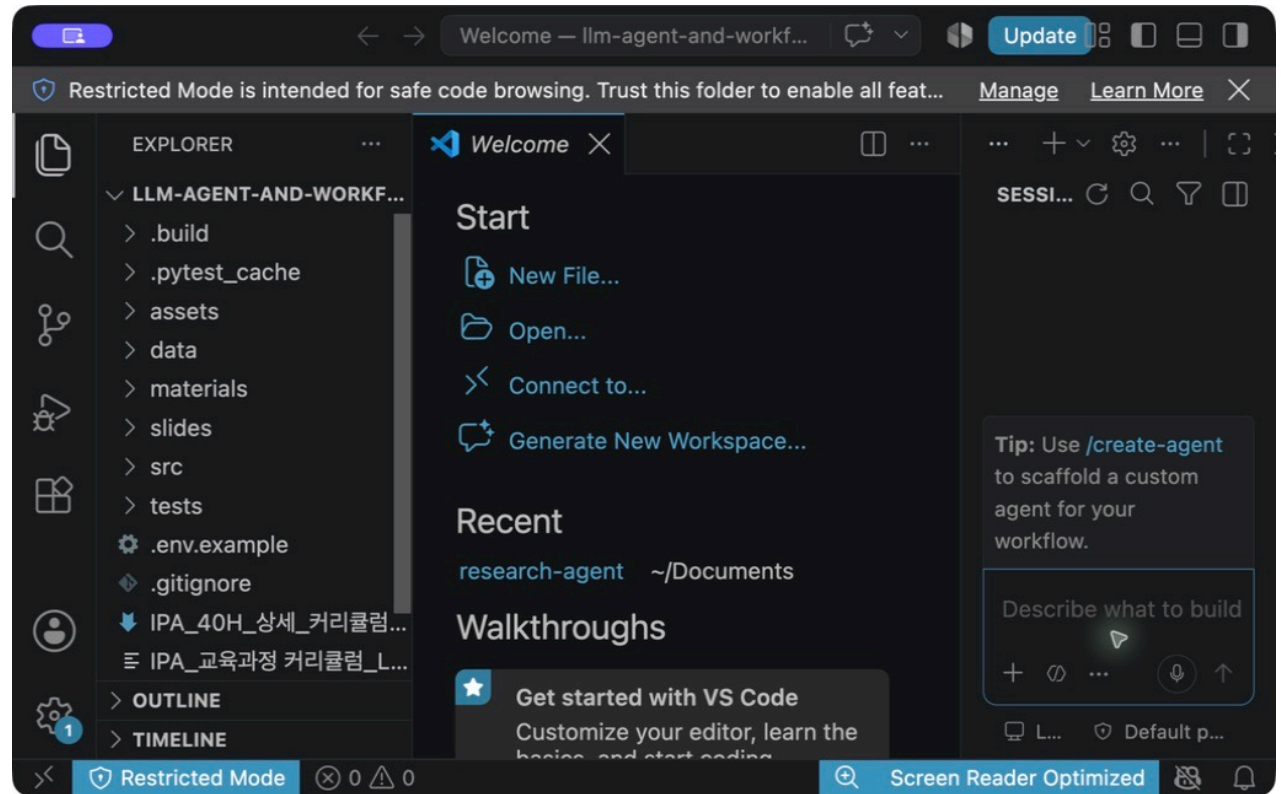


Anthropic Claude Code 공식 Overview

실제 VS Code 저장소 · 수업의 출발 화면

이 화면에서 딱 볼 것

- src·tests·materials를 먼저 찾기
- .env는 열지 않기
- 강사와 같은 폴더 구조 확인
- 실습 시작 화면으로 사용

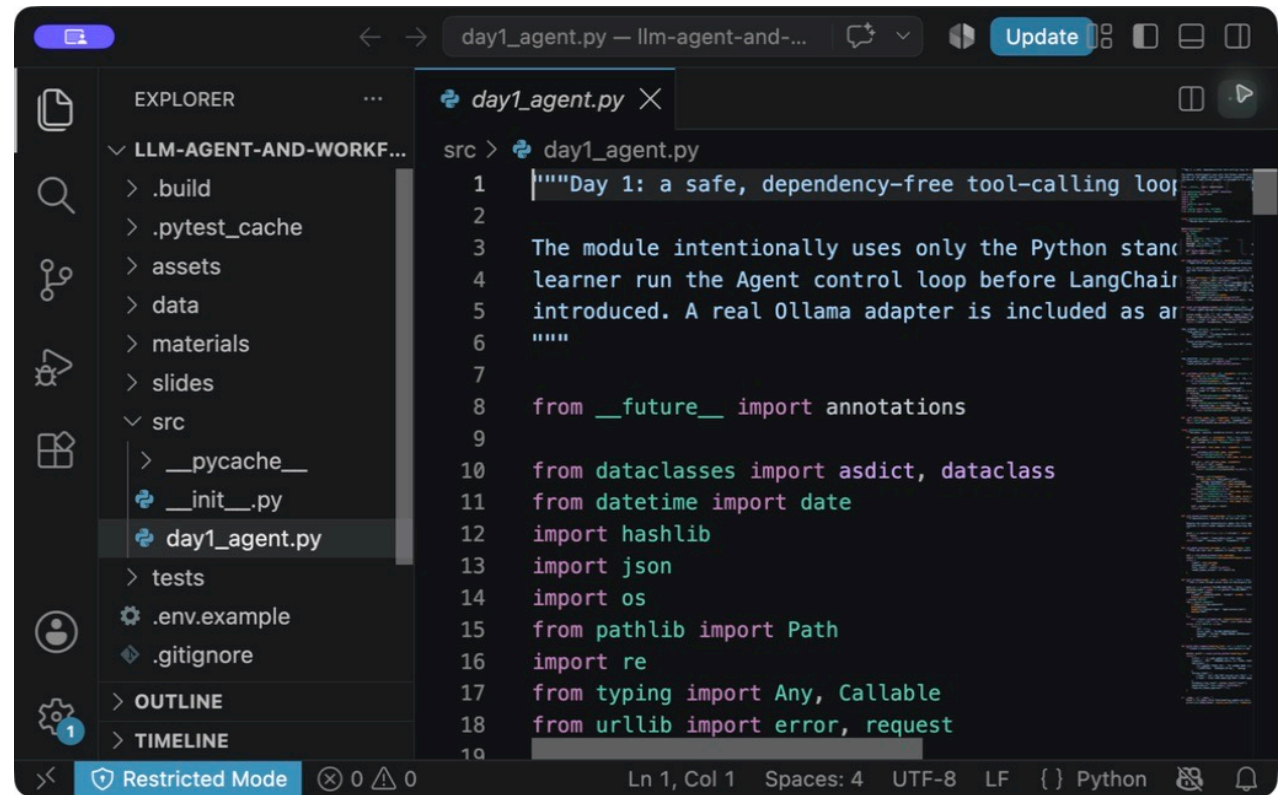


로컬에서 연 강의용 저장소

실제 Agent 코드 · 표준 라이브러리부터

이 화면에서 딱 볼 것

- 처음에는 framework 없이 제어 루프
- 읽기 도구만 허용
- 오류를 결과 객체로 정규화
- Ollama는 선택 adapter



```
src > day1_agent.py
1  """Day 1: a safe, dependency-free tool-calling loop
2
3  The module intentionally uses only the Python standard
4  learner run the Agent control loop before LangChain
5  introduced. A real Ollama adapter is included as an
6  """
7
8  from __future__ import annotations
9
10 from dataclasses import asdict, dataclass
11 from datetime import date
12 import hashlib
13 import json
14 import os
15 from pathlib import Path
16 import re
17 from typing import Any, Callable
18 from urllib import error, request
19
```

VS Code에서 src/day1_agent.py 연

작업 요청을 먼저 spec으로 고정

코드 전체보다 경계 한 줄을 먼저 봅니다

- 목표를 한 문장으로
- 입력·출력 형식
- 허용 행동
- 중단과 완료 증거

```
goal: 안전한 파일 읽기 Agent
input: 공개 txt/md
allowed_tools: [read_public_text]
stop: validation_error | success
evidence: pytest + output json
```


Agent의 최소 반복 구조

코드 전체보다 경계 한 줄을 먼저 봅니다

- 판단과 실행을 분리
- 매 step 결과 관찰
- 무한 반복 금지
- state로 감사 가능

```
while state.status == "RUNNING":  
    call = planner(state)  
    checked = validate(call)  
    result = execute(checkered)  
    state = observe(state, result)  
return state
```

Day 1 완료조건을 명령어로 표현

코드 전체보다 경계 한 줄을 먼저 봅니다

- 실행 결과
- 테스트
- 변경 범위
- 설명 가능한 diff

```
python3 -m src.day1_agent
python3.12 -m pytest -q
git status --short
git diff -- src tests

# success: 25 passed
```

저장소가 말해주는 역할 분리

코드 전체보다 경계 한 줄을 먼저 봅니다

- 코드와 데이터 분리
- 노트북과 모듈 분리
- 테스트를 1급 산출물로
- 장표와 파일명 연결

```
src/      # 실행 코드
tests/    # 실패·정상 증거
materials/ # 따라하기 notebook
data/     # 공개·합성 입력
slides/   # 수업 화면
```

강사 회의음성 Demo · 실패해도 수업은 계속

코드 전체보다 경계 한 줄을 먼저 봅니다

- 오디오가 있으면 Local STT
- 실패하면 같은 60초 transcript
- 사전 생성 JSON 즉시 확인
- 자동 메일 false·사람 승인 true

```
python3 -m src.meeting_demo ₩
--audio data/demo_meeting.wav ₩
--transcript data/demo_meeting_transcript.txt ₩
--out output/day1-demo

# outputs
# transcript.json + meeting_result.json
```

Ideation 1 · 강사 Demo를 보고 내 업무를 고릅니다

06 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 강사가 준비한 짧은 회의 음성을 코드에 넣습니다.
- 2 전사문·회의 요약·할 일 JSON이 만들어지는 화면을 봅니다.
- 3 자동 메일 발송이 차단되는 결과를 확인합니다.
- 4 같은 형식으로 내가 자동화할 일을 한 문장으로 적습니다.

예시 | 강사 시연 | demo_meeting.wav → transcript.json → meeting_result.json / 자동 메일 발송은 하지 않음

완료 증거 | 입력 1개 + 결과 1개 + 금지 행동 1개

Ideation 2 · 오늘 사용할 예시를 혼자 정합니다

05 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 재직자는 회사명·고객 정보를 뺀 반복 업무를 고릅니다.
- 2 구직자는 제공된 공개·합성 예시 중 하나를 고릅니다.
- 3 입력 자료와 원하는 결과를 한 줄씩 적습니다.
- 4 이후 소프트웨어 실습에서 같은 예시를 계속 사용합니다.

예시 | 회의 음성 → 결정·담당자·기한이 있는 회의 기록

완료 증거 | 내가 사용할 예시 한 문장

설계 워크숍 1 · 최종 데모 역산

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 최종 입력 한 건을 정한다.
- 2 보여줄 출력 JSON을 그린다.
- 3 사람이 승인할 지점을 표시한다.
- 4 LangSmith에서 보고 싶은 실패를 적는다.

완료 증거 | 입력→출력→승인 sketch

실행 확인 1 · 환경 신호등

06 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 Python·VS Code·Git 설치 여부를 표시한다.
- 2 관리자 권한 여부를 표시한다.
- 3 RAM·디스크 제약을 적는다.
- 4 GREEN/YELLOW/RED lane을 선택한다.

완료 증거 | 개인 환경 진단표

실패 1 · 문제 범위가 너무 크다

현업에서 자주 깨지는 방식

- 전사·요약·메일·ERP를 한 번에
- 사용자가 여러 명
- 정답 기준 없음
- 데모 입력도 미정

제가 먼저 고치는 순서

- 한 사용자·한 입력
- 한 구조화 출력
- 한 승인 행동
- 한 개의 golden example

실패 2 · 실제 회사 데이터를 가져온다

현업에서 자주 깨지는 방식

- 고객명·이메일 포함
- 비공개 회의 업로드
- 회사 repo token 사용
- 교육 계정에 trace 전송

제가 먼저 고치는 순서

- 공개·합성 데이터
- 이름·금액 범주화
- sandbox repo
- 민감 필드 masking

실패 3 · 모델 선택부터 시작한다

현업에서 자주 깨지는 방식

- 벤치마크 순위만 비교
- 큰 모델 다운로드 대기
- 문제 정의 지연
- 실패 조건 미정

제가 먼저 고치는 순서

- fixture로 흐름 먼저
- 작은 모델은 선택
- 계약과 test 우선
- provider 교체 가능

재직자 · 현업 PoC 승인 관점

현업 질문

- 어떤 시간을 줄이나?
- 어디서 사람이 승인하나?
- 어떤 데이터가 나가나?
- 장애 시 누가 복구하나?

남길 증거

- before/after 시간 가설
- approval policy
- data boundary
- 운영 runbook

Gate 1 · 범위를 줄일 수 있습니까?

상황: “회의 업무를 전부 자동화하자”는 요청을 받았습니다. 오늘 만들 수 있는 한 조각으로 잘라봅시다.

1 사용자 한 명과 입력 한 건을 무엇으로 정할까?

2 AI가 하면 안 되는 행동을 어디까지 막을까?

3 완료를 증명할 파일·test·demo는 무엇일까?

09:50-10:40

1일차 2차시

Tool Calling과 실행 권한

09:50-10:05 강의 · 10:05-10:15 코드 시연 · 10:15-10:35 소프트웨어 실습 · 10:35-10:40 확인

지금 배우는 내용	이 차시에서 이해할 것	함께 챙길 내용
Tool schema · 허용 목록 · 실행 전 검사	모델의 제안과 실제 실행 권한은 다르다	허용할 도구와 막을 행동

모델의 JSON은 아직 실행이 아닙니다

모델의 제안을 실제 행동으로 바꾸기 전에 누가 마지막으로 검사하는가?

요청

tool name과 arguments를 분리

계약

required·type·추가 필드를 검사

권한

allowlist와 workspace 경계를 확인

실행

registry에 등록된 함수만 호출

기록

result와 audit event를 함께 남김

실행 전에 알아야 할 네 가지 계약

용어	쉽게 말하면	오늘 쓰는 곳
Tool Calling Structured Tool Request	모델이 자유문장 대신 도구 이름과 인자를 구조화해 요청하는 방식이다.	<code>read_public_text(path=...)</code> 요청을 만들고 실행기는 따로 검증한다.
Schema Data Contract	입력과 출력에 어떤 필드가 필요하고 어떤 타입인지 정한 계약이다.	<code>path</code> 는 문자열, <code>action_items</code> 는 배열, <code>due</code> 는 날짜 또는 <code>null</code>
Validation Contract Check	실행 전에 입력이 Schema와 정책을 지키는지 검사하는 단계이다.	허용 확장자가 <code>txt/md</code> 가 아니면 <code>validation_error</code> 로 중단
Side Effect External Change	파일 쓰기·메일 발송·DB 변경처럼 외부 상태를 바꾸는 행동이다.	회의 요약을 Slack에 게시하는 행동은 승인 뒤 한 번만 실행한다.

Tool Calling은 실행 권한의 계약

모델의 JSON은 제안일 뿐, 실행 허가는 애플리케이션이 결정한다.

- name·description
- arguments schema
- allowlist·policy

권한 설계의 핵심 | Tool schema가 친절할수록 모델은 똑똑해 보이고, allowlist가 단단할수록 제품은 안전해진다.

Schema는 문서이자 방화벽

필수 필드·타입·열거값·추가 필드 금지를 코드로 고정한다.

- required
- type·enum
- additional properties

Registry가 실행 가능한 세계를 제한한다

모델이 아는 도구와 실제로 등록된 도구를 분리한다.

- TOOL_SCHEMAS
- TOOL_REGISTRY
- unknown tool 차단

최소 권한은 기능을 줄이는 설계

범용 shell보다 목적별 좁은 도구가 더 안전하고 평가하기 쉽다.

- read_public_text
- 확장자 제한
- workspace 경계

권한 설계의 핵심 | 오류 메시지도 제품 계약이다. 복구할 수 없는 오류는 관측되지 않은 오류다.

Validation은 실행 전에 끝낸다

누락·타입·예상 밖 인자는 side effect 전에 거부한다.

- missing
- unexpected
- wrong type

오류도 정상적인 결과 형식이다

traceback 대신 `ok·error_code·message`를 downstream에 전달한다.

- `VALIDATION_ERROR`
- `NOT_FOUND`
- `POLICY_BLOCKED`
- `TOOL_RUNTIME_ERROR`

Read와 Write는 위험도가 다르다

외부 쓰기는 승인·중복 방지·감사 로그가 추가로 필요하다.

- read: 자동 가능
- draft: 검토
- publish: 승인 필수

Audit event가 설명 가능성을 만든다

누가 무엇을 요청했고 무엇이 차단됐는지 구조화해 남긴다.

- input hash
- tool·args
- decision·result
- timestamp

권한 설계의 핵심 | 외부 쓰기 권한은 기능이 아니라 책임의 이전이다.

Tool 호출의 4단계

01

PLAN

도구·인자 제안

02

VALIDATE

schema·policy

03

EXECUTE

등록 함수만

04

OBSERVE

결과·오류 기록

Tool Prompt와 Schema를 만드는 순서

01

SYSTEM

역할·허용·금지 행동

02

TASK

입력과 원하는 결과

03

TOOLS

name·설명·인자 schema

04

EXAMPLE

정상 호출·차단 호출

파일 도구의 경로 검증

01

JOIN

workspace+path

02

RESOLVE

절대경로 정규화

03

CHECK

root 하위인지

04

ALLOW

확장자·존재

오류 분류 결정

01

정책

재시도 없음

02

사용자

입력 수정

03

일시

제한 retry

04

미지

중단·trace

테스트 설계 순서

01

정상

happy path

02

경계

최소·최대

03

공격

path traversal

04

중복

cached result

‘실행 허가’가 생기는 화면을 순서대로 봅니다

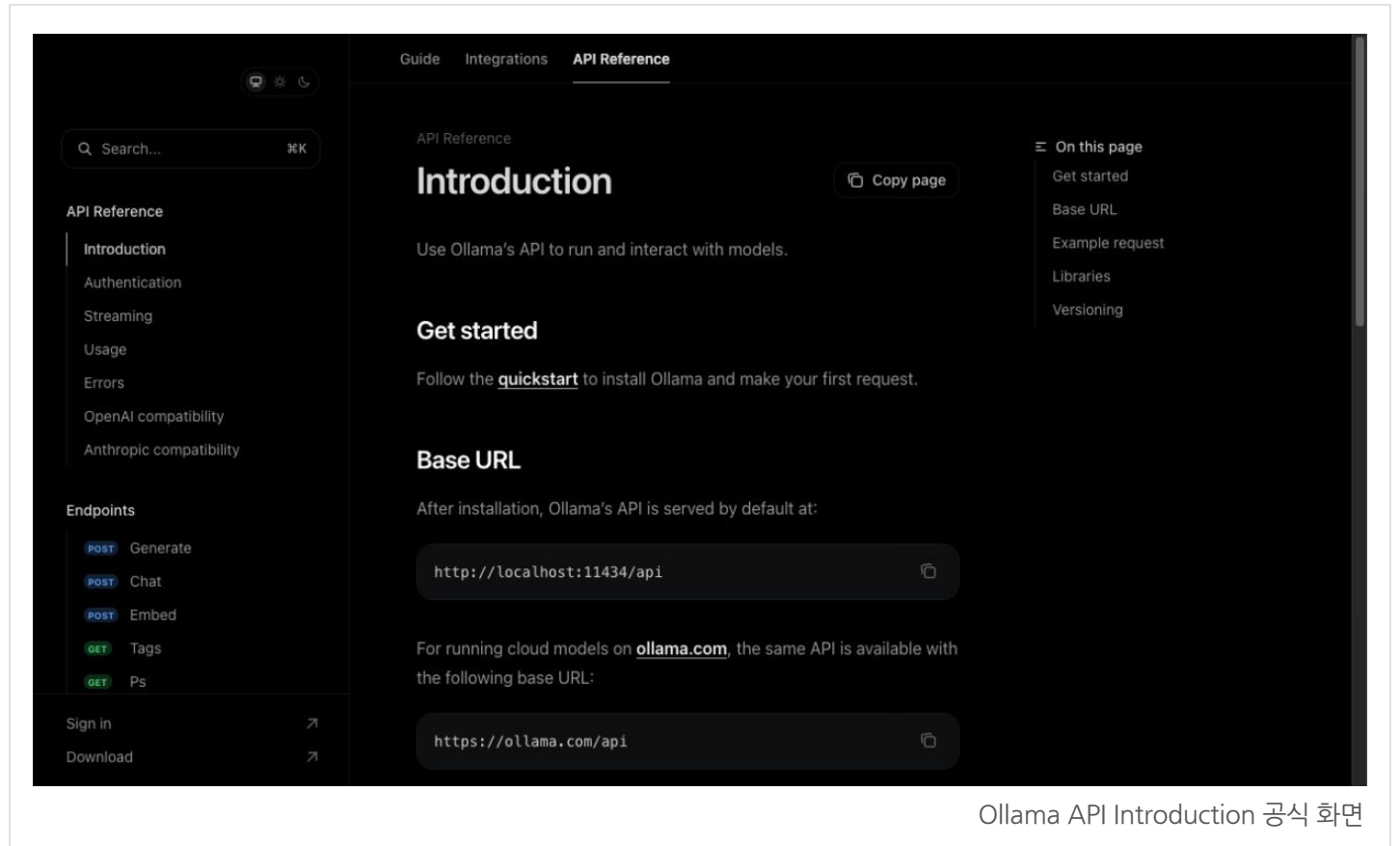
API 형식 → 인증 경계 → test 규칙 → 실제 통과 화면

01	요청·응답 형식 확인	base URL·payload
02	인증을 코드 밖으로 분리	환경 변수·최소 권한
03	실패를 test 함수로 고정	assert·error code
04	정상·예외를 한 번에 실행	25 passed

Ollama API · 호출 형식을 공식 문서로 확인

이 화면에서 딱 볼 것

- base URL 확인
- request/response 형식
- timeout·stream 선택
- adapter 뒤에 감추기

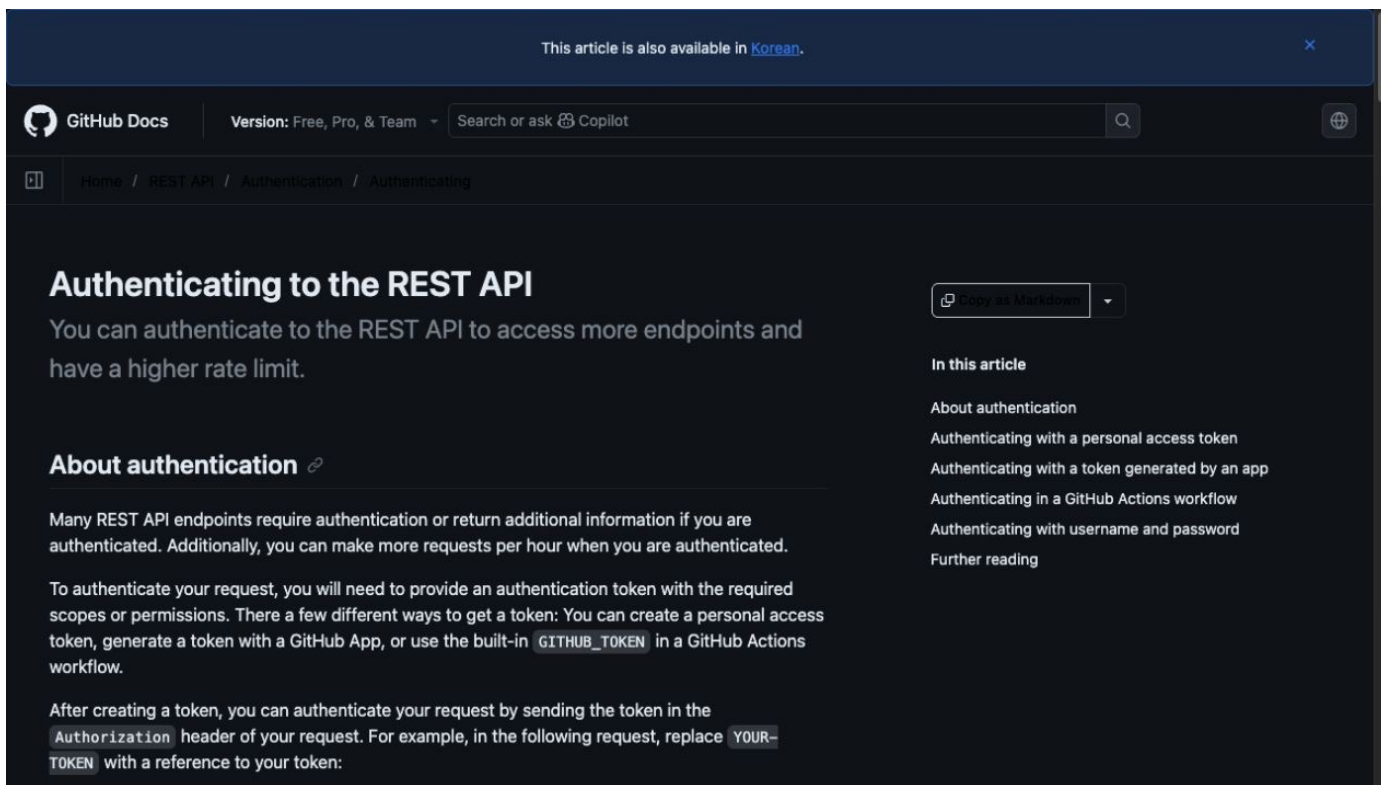


Ollama API Introduction 공식 화면

GitHub REST · 인증은 코드와 분리

이 화면에서 딱 볼 것

- token은 환경 변수
- 학생 필수 경로는 dry-run
- 최소 권한 확인
- 화면에 secret 노출 금지

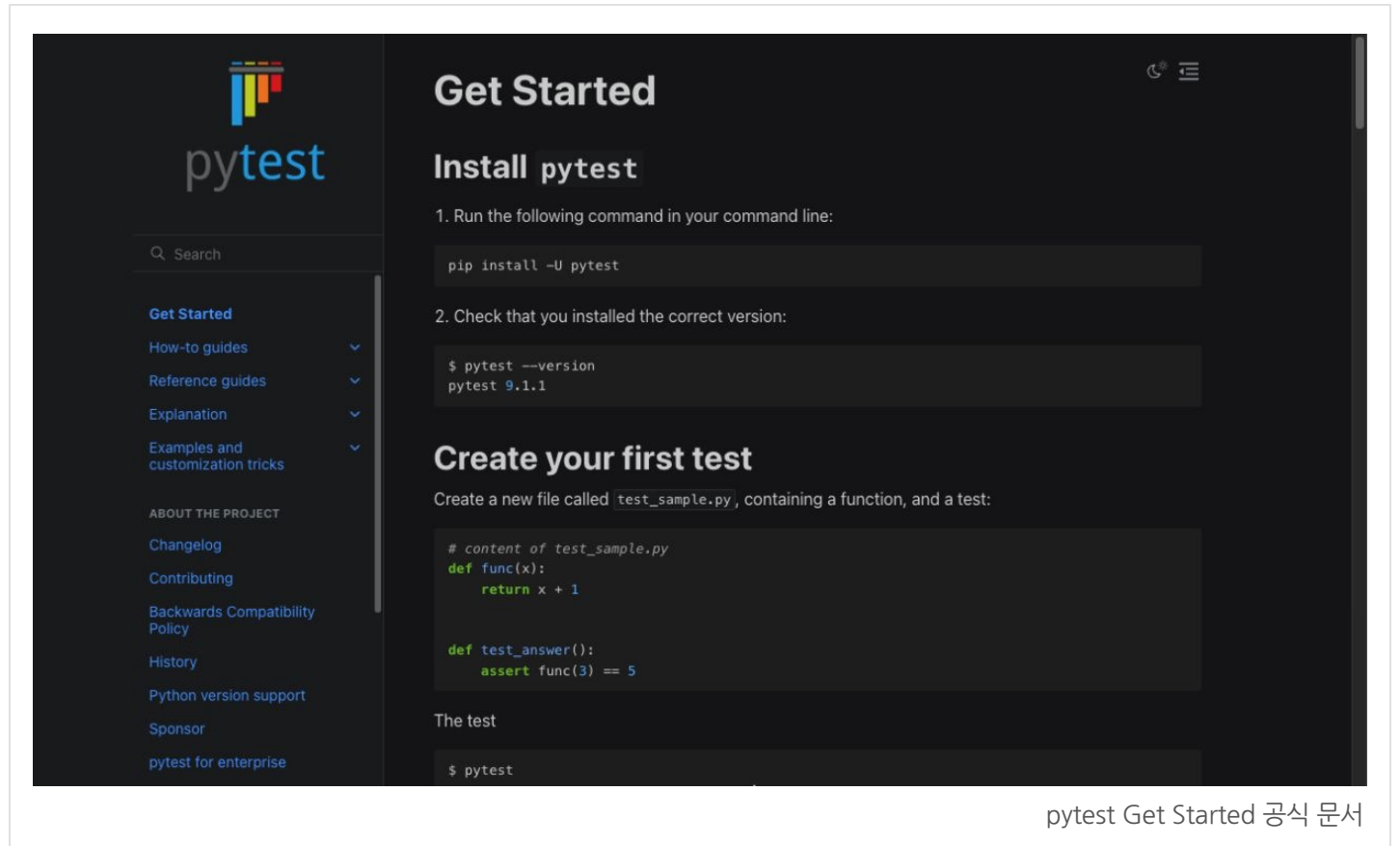


GitHub REST authentication 공식 문서

pytest · 오류를 재현 가능한 계약으로

이 화면에서 딱 볼 것

- test_ 함수 규칙
- assert로 계약 고정
- 정상·예외 함께
- 짧은 명령으로 반복



도구 schema · 허용 입력만

코드 전체보다 경계 한 줄을 먼저 봅니다

- 한 도구 한 책임
- 필수 인자 명시
- 설명은 모델용
- 실행 함수와 분리

```
TOOL_SCHEMAS = {  
    "read_public_text": {  
        "required": {"path": str},  
        "description": "공개 txt/md 읽기"  
    }  
}
```

예상 밖 인자를 차단

코드 전체보다 경계 한 줄을 먼저 봅니다

- 누락 검사
- 추가 필드 금지
- 실행 전 차단
- 오류 코드로 변환

```
required = schema["required"]  
missing = [k for k in required if k not in args]  
unexpected = set(args) - set(required)  
if missing or unexpected:  
    raise ToolValidationError()
```

경로 이탈을 막는 guard

코드 전체보다 경계 한 줄을 먼저 봅니다

- ../ 정규화 후 검사
- 절대경로 비교
- symlink 고려
- 테스트로 공격 재현

```
root = workspace.resolve()
candidate = (root / path).resolve()
if root != candidate and root not in candidate.parents:
    raise ToolValidationError("outside workspace")
```

오류를 공통 result로 정규화

코드 전체보다 경계 한 줄을 먼저 봅니다

- exception 경계
- 안정된 error_code
- 사용자 메시지
- trace에는 상세 stack

```
return ToolResult(  
    ok=False,  
    tool=tool_name,  
    error_code="POLICY_BLOCKED",  
    message=str(exc)  
)
```

Tool Calling Prompt의 최소 구조

코드 전체보다 경계 한 줄을 먼저 봅니다

- 역할과 금지 행동
- 입력·목표
- 도구 설명·schema
- 정상 예시 한 건

```
SYSTEM: 허용된 도구만 사용한다. 메일은 보내지 않는다.  
TASK: 회의 문서를 읽고 할 일을 찾는다.  
TOOLS: read_public_text(path: str)  
OUTPUT: 필요한 경우 tool call JSON만 반환한다.  
EXAMPLE: {"name":"read_public_text","arguments":{"path":"data/meeting.txt"}}
```

설계 워크숍 1 · 도구 한 개를 schema로

07 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 업무 동사를 한 개 고른다.
- 2 필수 인자를 1-2개로 제한한다.
- 3 타입과 허용값을 적는다.
- 4 정상 호출 예시를 JSON으로 쓴다.

완료 증거 | tool schema 1개

소프트웨어 실습 1 · 잘못된 Tool Call 실행

09 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 unknown tool을 실제로 호출한다.
- 2 필수 인자를 지운 호출을 실행한다.
- 3 예상 밖 인자를 넣어 실행한다.
- 4 세 error_code를 결과 객체에서 확인한다.

완료 증거 | 실패 ToolResult 3개

소프트웨어 실습 2 · workspace guard 실행

09 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 정상 상대경로로 함수를 실행한다.
- 2 ../를 포함한 경로로 다시 실행한다.
- 3 .py 확장자를 넣어 실행한다.
- 4 어느 guard에서 차단됐는지 test와 비교한다.

완료 증거 | 정상 1 + 차단 2

소프트웨어 실습 3 · ToolResult 계약 검증

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 성공·실패 호출을 같은 셀에서 실행한다.
- 2 두 result의 공통 필드를 비교한다.
- 3 needs_human_review 값을 확인한다.
- 4 결과 JSON을 파일로 저장한다.

완료 증거 | 공통 result JSON

실패 1 · 모델 JSON을 바로 실행

현업에서 자주 깨지는 방식

- schema 검증 생략
- 임의 도구 이름
- 추가 인자 허용
- side effect 직행

제가 먼저 고치는 순서

- plan은 제안
- validator가 허가
- registry 함수만
- 쓰기 전 preview

실패 2 · 범용 shell 도구 제공

현업에서 자주 깨지는 방식

- rm·curl까지 가능
- 명령 문자열 해석
- 권한 범위 불명
- 평가 불가능

제가 먼저 고치는 순서

- 목적별 함수
- typed arguments
- 최소 파일 범위
- 정상·실패 test

실패 3 · 모든 오류를 retry

현업에서 자주 깨지는 방식

- 정책 오류 반복
- 비용·시간 증가
- 중복 쓰기
- 원인 은폐

제가 먼저 고치는 순서

- 오류 class 분류
- 일시 오류만 retry
- max attempts
- 중단 후 사람 검토

재직자 · Tool 권한 승인표

자동 허용

- 공개 문서 읽기
- 로컬 parsing
- schema validation
- draft 저장

사람 승인

- 메일·Issue 게시
- 고객정보 접근
- 결제·삭제
- 비공개 repo 쓰기

Gate 2 · 이 Tool Call을 실행할까요?

상황: 등록되지 않은 send_email 도구에 예상 밖의 admin=true 인자가 들어왔습니다.

1

실행·차단·사람 검토 중 무엇을 선택할까?

2

downstream에 돌려줄 error_code는 무엇일까?

3

감사 event에 어떤 요청·판단·결과를 남길까?

10:40-11:30

1일차 3차시

한국어 데이터와 PBL 사례

10:40-10:54 강의 · 10:54-11:04 Prompt-데이터 시연 · 11:04-11:25 자료 수집-데이터 확인 · 11:25-11:30 저장

지금 배우는 내용	이 차시에서 이해할 것	함께 챙길 내용
회의 데이터 · 담당자·기한·원문 근거	자연스러운 요약보다 근거가 남아야 한다	입력·결과·근거 예시

한국어 요약보다 먼저 근거 단위를 만듭니다

요약이 그럴듯해도 원문 근거를 찾지 못하면 업무 결과라고 부를 수 있는가?

출처

공식 원문과 이용 조건을 확인

단위

발화를 segment와 timestamp로 분리

민감성

개인정보와 회사 정보를 비식별

정답

결정·담당자·기한·근거를 연결

고정

대표 실패를 Golden Dataset에 추가

한국어 데이터를 읽는 네 가지 단위

용어	쉽게 말하면	오늘 쓰는 곳
Transcript Speech as Text	음성 내용을 시간 순서대로 문자로 옮긴 기록이다.	회의 발언과 타임스탬프를 함께 저장한다.
Segment Timed Utterance	Transcript를 시작·종료 시각이 있는 짧은 발화 단위로 나눈 것이다.	41.2-48.7초의 ‘금요일까지 초안’ 발화
Golden Dataset Reference Examples	정답 또는 검토 기준이 붙은 소량의 대표 예시 모음이다.	정상 회의·무음·모호한 기한·근거 누락 5건
Evidence Source Grounding	결론이나 Action Item이 어디에서 나왔는지 가리키는 원문 근거이다.	Action Item마다 segment_id를 연결한다.

PBL은 기술 목록이 아니라 사용자 문제에서 시작

STT·LLM·LangGraph를 모두 쓰되, 한 사용자의 한 업무 흐름에 묶는다.

- 사용자·상황
- 반복 입력
- 원하는 결정·행동

한국어 업무 데이터의 함정 | 한국어 회의록에서 가장 비싼 오류는 문법이 아니라 사람·숫자·날짜 오류다.

K-Work Copilot의 공통 입력

강사는 17분 합성 회의 전체를 보여주고, 수강생은 같은 파일의 2-3분 구간부터 검증한다.

- meeting_sample_ko_12min.wav
- timestamp transcript
- expected JSON·Dataset Card

한국어는 띄어쓰기보다 업무 맥락이 어렵다

존칭·생략·동음어·숫자·날짜·담당자 모호성을 별도 오류로 본다.

- 화자 생략
- “다음 주” 기준일
- 제품명·약어
- 책임자 추론 금지

실제 데이터처럼 보이는 최소 조건

자연스러운 맥락·역할·결정·마감이 있으면서 개인 식별정보는 없어야 한다.

- 가상 조직
- 역할명
- 합성 금액 범주
- 공개 가능한 업무 관계

한국어 업무 데이터의 함정 | 공개 데이터는 무료이지만, 사용할 수 있는 맥락까지 무료로 주지는 않는다.

데이터 공개와 사용 허가는 다르다

웹에서 보인다는 사실만으로 재배포·수업 배포가 허용되지는 않는다.

- 이용 조건
- 다운로드·신청
- 재배포 범위
- 출처 기록

Dataset Card가 실습을 재현하게 한다

출처·버전·샘플링·민감성·변환을 한 페이지에 남긴다.

- source URL
- collected_at
- license·terms
- preprocessing·limitations

Transcript schema가 STT와 LLM을 연결

텍스트만 넘기지 말고 segment·time·speaker·quality flag를 유지한다.

- start·end
- speaker
- text
- no_speech·language

Evidence가 요약의 근거를 고정한다

결정과 Action Item마다 원문 segment ID를 연결한다.

- claim
- evidence_ids
- confidence
- needs_review

한국어 업무 데이터의 함정 | Golden Dataset은 정답 모음이 아니라 팀이 합의한 품질의 경계다.

공개 데이터 선택 4단계

01

DISCOVER

공식 원자료 찾기

02

VERIFY

이용 조건 확인

03

SAMPLE

소량·목적 적합

04

DOCUMENT

출처·변환 기록

한국어 업무 Prompt의 기본 구조

01	02	03	04
ROLE	INPUT	OUTPUT	RULE
회의 기록 보조자	segment·기준일	summary·decision·action	추론 금지·근거 필수

비식별 처리 순서

01

DETECT

이름·연락처·ID

02

REPLACE

역할·범주

03

REVIEW

문맥 재식별

04

LOG

변환 규칙

Golden set 만드는 순서

01

TRANSCRIBE

사람 정답

02

ANNOTATE

결정·행동

03

LINK

evidence segment

04

AGREE

2인 합의

PBL canvas 네 칸

01

USER

누가 쓰나

02

PAIN

무엇이 누락되나

03

FLOW

AI와 사람 역할

04

PROOF

무엇으로 좋아졌나

데이터보다 먼저 출처와 발화 구조를 확인합니다

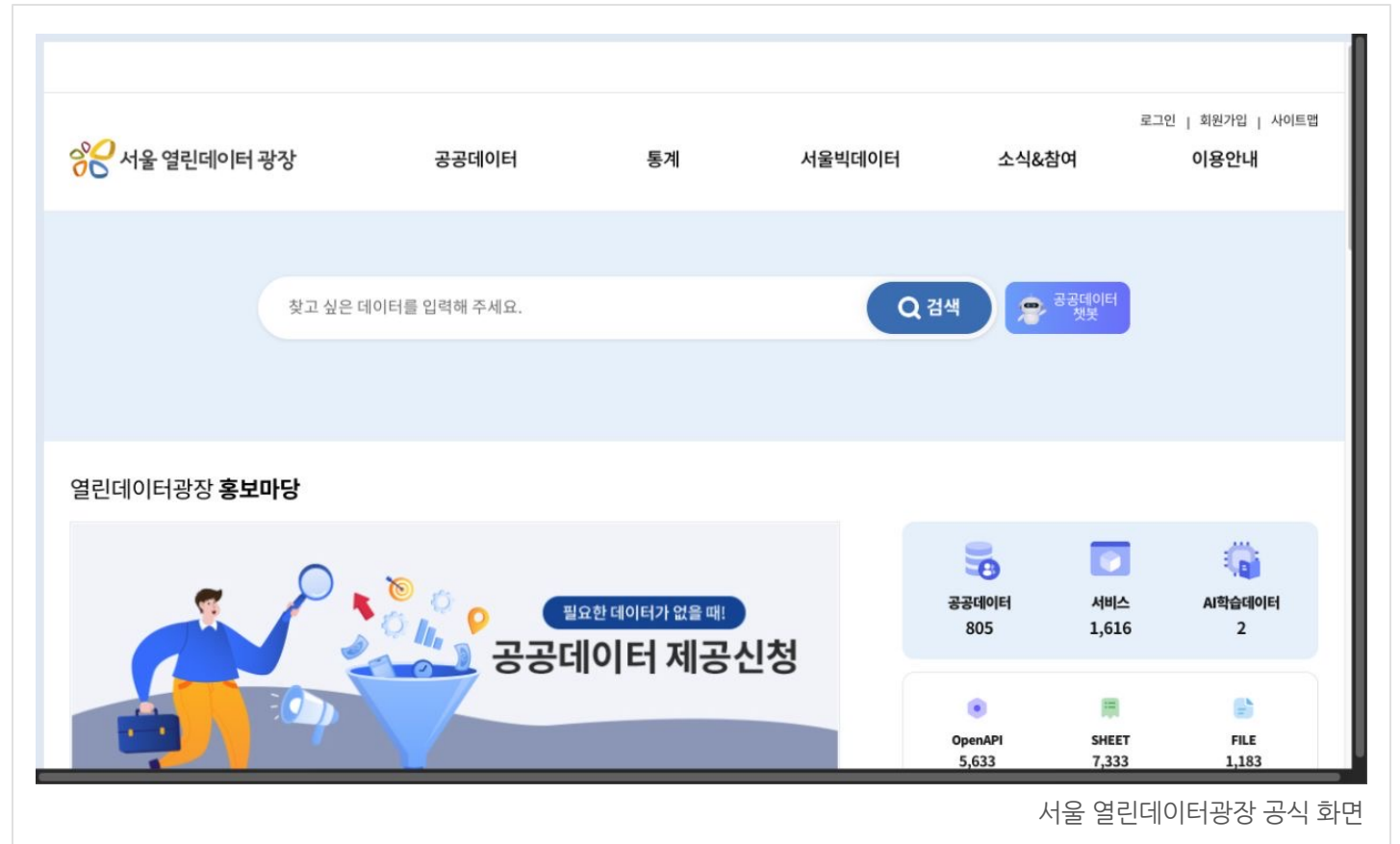
공공 데이터 → 말뭉치 → 회의록 → Managed 전사 화면

도구·서비스	도메인·역할	이번 강의에서 쓰는 이유	유사 도구와의 차이
서울 열린데이터광장	공공·행정 데이터	한국어 공개 데이터의 출처와 갱신 주기를 확인	회의 발화 데이터는 별도 가공 필요
모두의 말뭉치	한국어 원자료	문장·대화 자료의 언어 특성을 이해	이용 신청과 재배포 조건 확인
국회 회의록	공식 회의 발화	화자·안전·발언 구조가 실제 업무와 가까움	목표 업무에 맞는 정답 라벨은 없음
Google Meet	Managed STT·노트	현업에 이미 있는 전사·요약 기능을 활용	구독·edition·관리자 정책 의존

서울 열린데이터광장 · 한국어 공공 데이터 탐색

이 화면에서 딱 볼 것

- 공식 제공처를 먼저
- 검색어·제공기관 기록
- 다운로드 형식 확인
- 소량 샘플부터



국립국어원 모두의 말뭉치 · 한국어 원자료

이 화면에서 딱 볼 것

- 신청·이용 조건 확인
- 말뭉치 종류와 목적 구분
- 원문 재배포 주의
- dataset card에 출처

언어정보나눔터 | 모두의 말뭉치

말뭉치 분류

배포 연도

국립국어원 모두의 말뭉치 화면

국회 회의록 빅데이터 · 실제 발화 구조

이 화면에서 딱 볼 것

- 발언자·회의 맥락
- 공개 회의 기록 사례
- 페이지·첨부 공개 범위 확인
- 학습용 소량 사례

The screenshot displays the NAIL National Assembly AI Library homepage. At the top, there's a navigation bar with the NAIL logo, the text '국회도서관 발언 빅데이터' (National Assembly Library Speech Big Data), and social media icons. A search bar is prominently featured with the placeholder text '검색어를 입력하세요. (발언자명, 발언 내용, 안건명, 회의명)' (Enter search terms. (Speaker name, Speech content, Agenda name, Session name)). Below the search bar, a message states: '국회·지방의회 회의록에 대한 발언자, 키워드 검색 및 다운로드, 인포그래픽 등 다양한 의정기록 데이터 서비스를 제공합니다.' (We provide various legislative record data services including speaker search, keyword search, and download, infographics, etc., for the National Assembly and local assembly session records). A large statistics section shows '회의록 건수' (Number of session records) as 39,370 and '발언 데이터셋' (Speech dataset) as 16,637,220. Below this, there are two main sections: '서비스 통계' (Service Statistics) with a sub-link '회의록수 | 발언 데이터셋' (Number of session records | Speech dataset), and '최근 회의록' (Recent session records) with a sub-link '[임시] 제22대 국회 제438회 제3차 교육위...' (Temporary 22nd National Assembly 438th 3rd Education Committee...). The date '2026-08-14' is also visible. At the bottom right, it says '국회 회의록 빅데이터 공식 화면' (Official National Assembly session record big data screen).

Google Meet 전사 · Managed STT의 현재 위치

이 화면에서 딱 볼 것

- 계정·edition 의존
- 한국어 지원 여부 확인
- 원문 보존·권한 정책
- 수업은 local 원리 중심

Google Meet 고객센터

문제를 설명해 주세요.

로그인

Google Meet

Google Meet 시작하기 | 화상 회의 시작 또는 참여하기 | Gemini in Meet | 회의 중 작업 | 회의 종료 후 | **기능 사용 가능 여부** | Google Meet 설정 | Google Meet에서 통화하기

Google Meet 프리미엄 기능 > Google Meet에서 스크립트 사용하기

Google Meet에서 스크립트 사용하기

중요:

- 스크립트 기능은 컴퓨터, 노트북 또는 Android 기기에서만 사용할 수 있습니다.
- 기본적으로 회의 스크립트가 사용 중지되어 있는 학생 라이선스의 Google Workspace for Education 이외에는 모든 Workspace 버전에 회의 스크립트가 기본적으로 사용 설정되어 있습니다. [스크립트 설정에 대해 자세히 알아보기](#)

다음 두 드라이브 모두에 공간이 충분한 경우에만 회의 스크립트를 녹화할 수 있습니다.

- Workspace 조직의 Google Drive
- 회의 호스트의 Drive

[스크립트가 있는 Google Workspace 버전 알아보기](#)

[지원되는 언어 알아보기](#)

Google Meet 프리미엄 기능

- Google Workspace를 위한 프리미엄 Meet 기능
- 화상 회의 녹화하기
- Google Meet에서 스크립트 사용하기**
- Google Meet에서 번역된 자막 사용하기
- Google Meet에서 소그룹 채팅방 사용하기
- Google Meet에서 Calendar 참석자 목록 보기
- Google Meet에서 참여자에게 질문하기
- Google Meet에서 설문조사 진행하기
- Google Meet에서 Slides 공동 발표하기

Google Meet transcript 도움말

회의 segment schema

코드 전체보다 경계 한 줄을 먼저 봅니다

- timestamp 유지
- speaker는 모르면 null
- 원문 text 보존
- 품질 flag 별도

```
segment = {  
  "id": "s12",  
  "start": 41.2,  
  "end": 48.7,  
  "speaker": "역할1",  
  "text": "금요일까지 초안을...",  
  "quality_flags": []  
}
```

Action Item에 evidence 연결

코드 전체보다 경계 한 줄을 먼저 봅니다

- task·owner·date
- 근거 segment
- confidence 정책
- 누락 시 review

```
action = {  
  "task": "FAQ 30건 정리",  
  "owner": "역할1",  
  "due_date": "2026-08-27",  
  "evidence_ids": ["s12"],  
  "confidence": 0.92  
}
```

모호한 날짜는 추론하지 않는다

코드 전체보다 경계 한 줄을 먼저 봅니다

- 기준일 필요
- null 허용
- 모호성 코드
- 사람에게 질문

```
if due_text in ["다음 주", "조만간"]:  
    return {  
        "due_date": None,  
        "needs_review": True,  
        "reason": "RELATIVE_DATE"  
    }
```


한국어 회의 Prompt · 모호하면 추론하지 않기

코드 전체보다 경계 한 줄을 먼저 봅니다

- 역할을 한 문장으로
- 금지 행동을 먼저
- 출력 schema 고정
- 원문 근거 요구

SYSTEM: 한국어 회의 기록 보조자

RULES:

- 원문에 없는 담당자·기한을 추론하지 않는다.
- 모호하면 null과 needs_review를 반환한다.

INPUT: segments[] + meeting_date

OUTPUT: MeetingResult JSON

EVIDENCE: 모든 결정·할 일에 evidence_ids를 연결한다.

Dataset card 최소 필드

코드 전체보다 경계 한 줄을 먼저 봅니다

- 출처
- 수집 시점
- 민감성
- 변환·제한

```
dataset:  
  source: official_url  
  collected_at: 2026-08-22  
  purpose: classroom_demo  
  contains_pii: false  
  transformations: [sampling, anonymize]  
  limitations: [synthetic_speakers]
```

자료 수집 1 · 한국어 사례 후보 3개

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 공식 데이터 사이트에서 검색어를 정한다.
- 2 사례 후보를 세 개 적는다.
- 3 업무 흐름과 가장 가까운 한 개를 고른다.
- 4 선택 이유를 한 문장으로 쓴다.

완료 증거 | 후보 3개 + 선택 1개

자료 수집 2 · 이용 조건 확인표

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 제공 기관과 URL을 기록한다.
- 2 다운로드·신청 필요 여부를 적는다.
- 3 재배포 가능 여부를 확인한다.
- 4 불확실하면 강사 fixture를 선택한다.

완료 증거 | source checklist

데이터 확인 1 · 상세 회의에서 근거 구간 찾기

10 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 meeting_sample_ko_12min.txt를 엽니다.
- 2 결정 1개와 정정 발언 1개를 찾습니다.
- 3 담당·기한 Action Item 2개를 찾습니다.
- 4 “다음 주 중”을 needs_review로 표시합니다.

완료 증거 | evidence segment 4개

소프트웨어 실습 1 · Golden JSON diff 실행

10 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 내 결과를 JSON으로 저장합니다.
- 2 Action Item 두 개에 evidence ID를 연결합니다.
- 3 기준 JSON과 diff 명령을 실행합니다.
- 4 다른 부분을 오류·허용 표현 차이로 나눕니다.

완료 증거 | 내 JSON + 기준 JSON diff

실패 1 · 데이터가 너무 크다

현업에서 자주 깨지는 방식

- 수십 시간 audio
- 수백 MB 전체 말뭉치
- 전처리만 수업 소진
- 정답 만들기 불가

제가 먼저 고치는 순서

- 2-3분 audio
- 10-30건 text
- golden 3-5건
- 확장 과제는 별도

실패 2 · 공개 페이지를 그대로 재배포

현업에서 자주 깨지는 방식

- 이용 조건 미확인
- 첨부 전체 공유
- 출처 누락
- 신청 계정 공유

제가 먼저 고치는 순서

- 공식 링크 제공
- 허용된 소량 샘플
- 출처·날짜 기록
- fixture 대체

실패 3 · 정답 없이 prompt를 튜닝

현업에서 자주 깨지는 방식

- 좋아 보이는 출력
- 기준 매번 변경
- 모델 비교 불가
- 오류 회귀 모름

제가 먼저 고치는 순서

- golden example
- rubric
- 고정 dataset
- experiment 비교

재직자 · 실제 업무를 안전하게 추상화

가져오지 않을 것

- 실명·고객번호
- 계약 금액
- 회사 URL
- 내부 전략

남길 관계

- 역할 간 의사결정
- 업무 단계
- 승인 조건
- 실패 비용

Gate 3 · 이 Action Item을 믿어도 될까요?

상황: 요약에는 “다음 주에 민지가 배포”라고 적혔지만 원문에는 담당자와 날짜가 명확하지 않습니다.

1

어떤 segment를 evidence로 다시 확인할까?

2

담당자·기한을 null로 둘지 사람에게 물을지 결정하자.

3

이 사례를 Golden Dataset에 어떤 실패 유형으로 남길까?

12:00-12:50

1일차 4차시

Python·Jupyter·Ollama 환경

12:00-12:08 강의 · 12:08-12:18 notebook 시연 · 12:18-12:42 설치·진단·test · 12:42-12:50 확인

지금 배우는 내용	이 차시에서 이해할 것	함께 챙길 내용
Library 설치 · Python 경로 · Ollama CLI·서버·모델 · 좁은 test	Ollama 설치와 모델 준비는 다른 상태다	설치 상태·Ollama 상태 JSON· 4 passed

같은 명령이 같은 Python을 가리켜야 합니다

설치가 실패해도 수업과 프로젝트를 계속할 수 있는 두 번째 경로가 있는가?

확인

OS·Python·Git 버전을 먼저 캡처

격리

프로젝트 .venv를 생성

선택

VS Code와 Notebook interpreter를 맞춤

보호

secret·대용량 파일을 Git 밖에 둬

복구

첫 commit과 fixture 경로를 남김

설치보다 먼저 구분할 네 가지 환경

용어	쉽게 말하면	오늘 쓰는 곳
Runtime Execution Environment	코드가 실제로 실행되는 Python·운영체제·패키지 환경이다.	<code>python3 --version</code> 과 실행 경로를 먼저 확인한다.
Interpreter Selected Python	VS Code나 Notebook이 사용할 특정 Python 실행 파일이다.	<code>.venv/bin/python</code> 을 VS Code Interpreter로 선택한다.
Virtual Environment Isolated Packages	프로젝트별 Python 패키지를 격리하는 폴더이다.	<code>python3 -m venv .venv</code> 후 같은 환경에서 설치·실행한다.
Repository Versioned Project Folder	코드·문서·데이터와 변경 이력을 함께 관리하는 프로젝트 폴더이다.	<code>src·tests·data·materials</code> 폴더를 하나의 저장소로 관리한다.

무료의 기준은 끝까지 직접 실행되는가

외부 LLM API 결제 없이 필수 Tool·Chain·Graph를 실행할 수 있어야 한다.

- Python·Jupyter
- fixture·local LLM
- 실행 결과 JSON

수업을 살리는 기준 | 환경 문제의 절반은 패키지가 아니라 '지금 어떤 Python을 쓰는가'에서 시작한다.

필수 경로는 notebook, 가속 경로는 Codex

모든 수강생은 준비된 코드를 직접 실행하고, Codex·PR은 먼저 끝낸 사람의 선택 확장으로 둔다.

- 필수: 04 notebook
- 선택: diff·PR
- 공통: test 확인

Python은 실행기, VS Code는 작업 공간

extension이 Python 자체를 설치해 주는 것은 아니다.

- Python interpreter
- VS Code editor
- Python·Jupyter extension

가상환경은 프로젝트의 재현 경계

전역 환경 대신 프로젝트별 dependency를 격리한다.

- `python -m venv`
- interpreter 선택
- `requirements.pyproject`

수업을 살리는 기준 | 첫 commit은 저장이 아니라 복구 지점을 만드는 일이다.

Ollama 준비는 세 칸으로 나눠 본다

앱 설치와 server 실행, model 다운로드는 서로 다른 상태다.

- cli_installed
- server_ready
- model_ready

Jupyter는 설명과 실행의 다리

한 셀 한 개념으로 결과를 보고, 모듈 코드는 src에 둔다.

- Markdown 설명
- Code 실행
- output 증거
- Restart & Run All

Secret은 코드·노트북·캡처 밖에 둔다

실제 값은 환경 변수, 저장소에는 이름만 있는 example을 둔다.

- .env.example
- .gitignore
- 화면 비식별

설치 셀은 없는 library만 채운다

상단 셀은 필수와 Ollama 선택 package를 분리하고, 선택 설치 실패는 fixture로 복구한다.

- requirements-day1
- requirements-local-llm-optional
- Restart Kernel

수업을 살리는 기준 | Harness Engineering의 출발은 좋은 프롬프트보다 깨끗한 작업공간이다.

4차시 실행 순서

01

OPEN

04 notebook

02

INSTALL

상단 library 셀

03

PROBE

CLI·server·model

04

TEST

4 passed

가상환경 준비

01

CREATE

`python -m venv`

02

ACTIVATE

OS별 명령

03

INSTALL

`requirements`

04

VERIFY

`python -m pip`

VS Code interpreter 선택

01

PALETTE

Command Palette

02

SELECT

Python interpreter

03

KERNEL

notebook kernel

04

CHECK

sys.executable

Ollama 상태 진단

01

CLI

명령 설치

02

SERVER

11434 응답

03

MODEL

qwen3:4b 존재

04

LANE

ollama:fixture

library 설치 실패 복구

01

READ

첫 오류

02

RETRY

상단 셀 한 번

03

RESTART

Kernel

04

FALLBACK

fixture

설치 화면은 ‘어디서 받는가’보다 ‘무엇을 확인하는가’가 중요합니다

Python → VS Code interpreter → Git identity → sandbox repository

01	—————	내 OS에 맞는 runtime 선택	버전·실행 경로
02	—————	editor가 쓸 Python 지정	.venv interpreter
03	—————	Git 작성자와 기본 상태 확인	identity·status
04	—————	교육용 저장소만 연결	sandbox remote

실제 notebook · Ollama 세 단계 진단

이 화면에서 딱 볼 것

- Ollama 0.32.15
- local server 준비
- model 상태 확인
- recommended lane

4차시 · Ollama 설치·서버·모델 진단

공식 안내:

- macOS 설치: <https://docs.ollama.com/macos>
- Quickstart: <https://docs.ollama.com/quickstart>
- 수업 모델: <https://ollama.com/library/qwen3:4b>

강의 전 설치한 뒤 터미널에서 확인합니다.

```
ollama --version
ollama run qwen3:4b
ollama list
qwen3:4b 는 약 2.5GB이므로 수업 시작 후 전체가 다운로드를 기다리지 않습니다. 강사는 전날 준비하고, 수강생은 모델이 없으면 fixture 경로로 수업을 계속합니다.
```

In [3]:

```
from src.ollama_tool_agent import probe_ollama

ollama_status = probe_ollama()
print(json.dumps(ollama_status, ensure_ascii=False, indent=2))
```

```
{
  "cli_installed": true,
  "cli_path": "/usr/local/bin/ollama",
  "server_ready": true,
  "version": "0.32.15",
  "models": [
    "qwen3:4b"
  ],
  "model_ready": true,
  "error_code": null,
  "detail": null,
  "recommended_lane": "ollama"
}
```

환경 상태에 따른 진행 경로

상태	이번 수업에서 할 일
CLI·서버·모델 준비	provider="ollama" 로 실제 모델 호출

04 notebook 실행 화면

실제 notebook · 전체 25 tests 통과

이 화면에서 딱 볼 것

- 25개 test
- [100%] 완료
- 첫 오류 복구
- 점심 전 checkpoint

5차시 · 점심 전 전체 회귀 test

새로 만든 Ollama 경로뿐 아니라 기존 Agent·LangChain·LangGraph·STT 계약이 함께 유지되는지 전체 Day 1 test를 실행합니다. 실패하면 마지막 줄보다 첫 번째 오류부터 복구합니다.

```
In [9]: full_test = subprocess.run(
        [sys.executable, "-m", "pytest", "-q"],
        cwd=ROOT,
        capture_output=True,
        text=True,
        check=False,
    )
    print(full_test.stdout.strip())
    assert full_test.returncode == 0, full_test.stderr

.....
25 passed in 0.33s [100%]
```

6차시 · LangChain LCEL Provider 교체

같은 transcript와 Pydantic schema를 유지하고 provider만 바꿉니다.

Prompt → fixture/Ollama → Pydantic Parser → Policy Validator

Ollama용 선택 패키지는 다음과 같이 설치합니다.

python -m pip install -r requirements-local-llm-optional.txt

```
In [10]: from src.langchain_lab import run_langchain_lab

transcript = (ROOT / "data/meeting_sample_ko_12min.txt").read_text(encoding="utf-8")
fixture_chain = run_langchain_lab(transcript, provider="fixture")
print(json.dumps({
    "provider_used": fixture_chain["provider_used"],
    "pipeline": fixture_chain["pipeline"],
    "checks": fixture_chain["checks"],
}, ensure_ascii=False, indent=2))

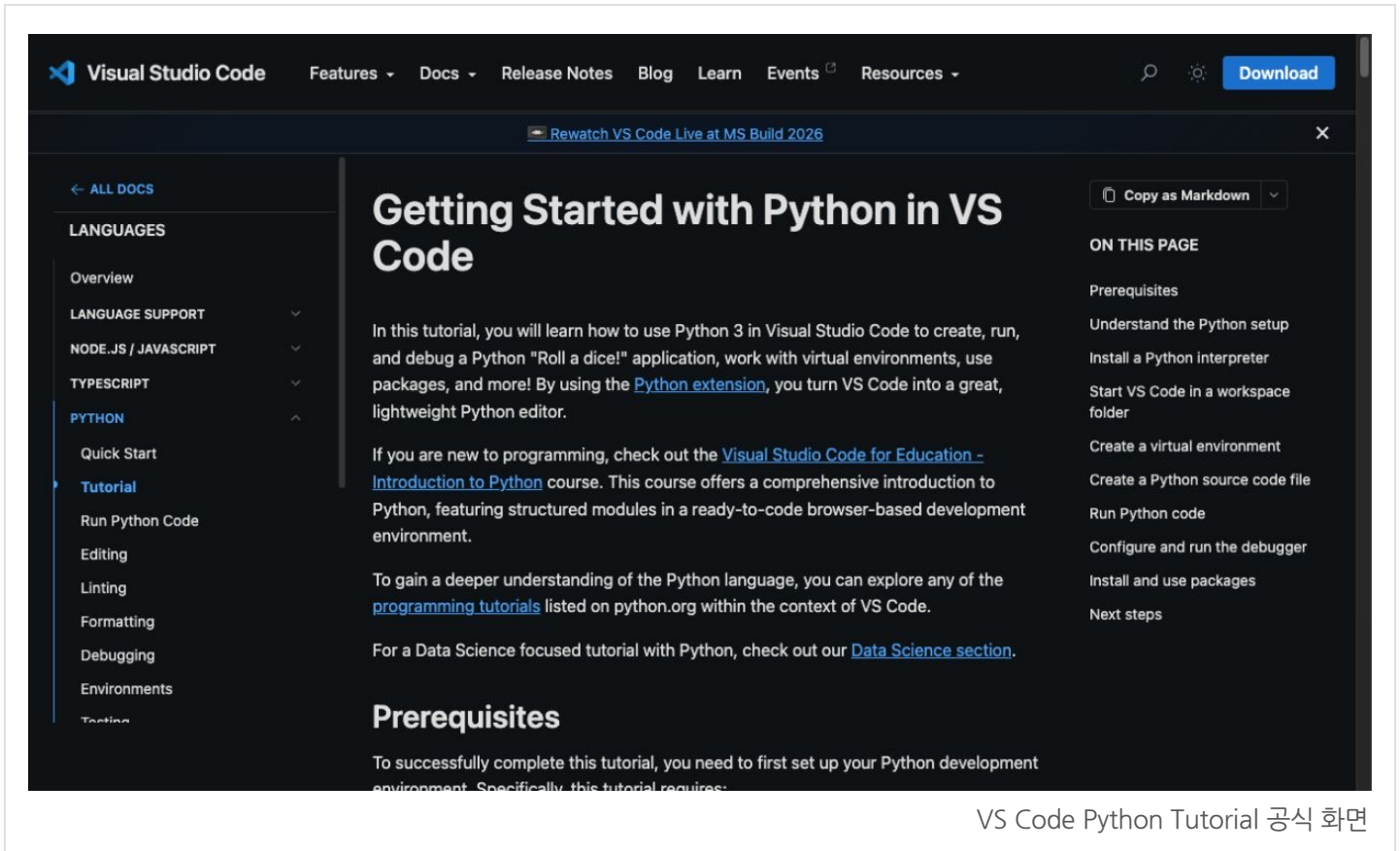
{
  "provider_used": "fixture",
  "pipeline": [
    "Ollama"
  ]
}
```

04 notebook 전체 회귀 test

VS Code Python 튜토리얼 · 세 구성요소

이 화면에서 딱 볼 것

- VS Code
- Python extension
- Python interpreter
- virtual environment

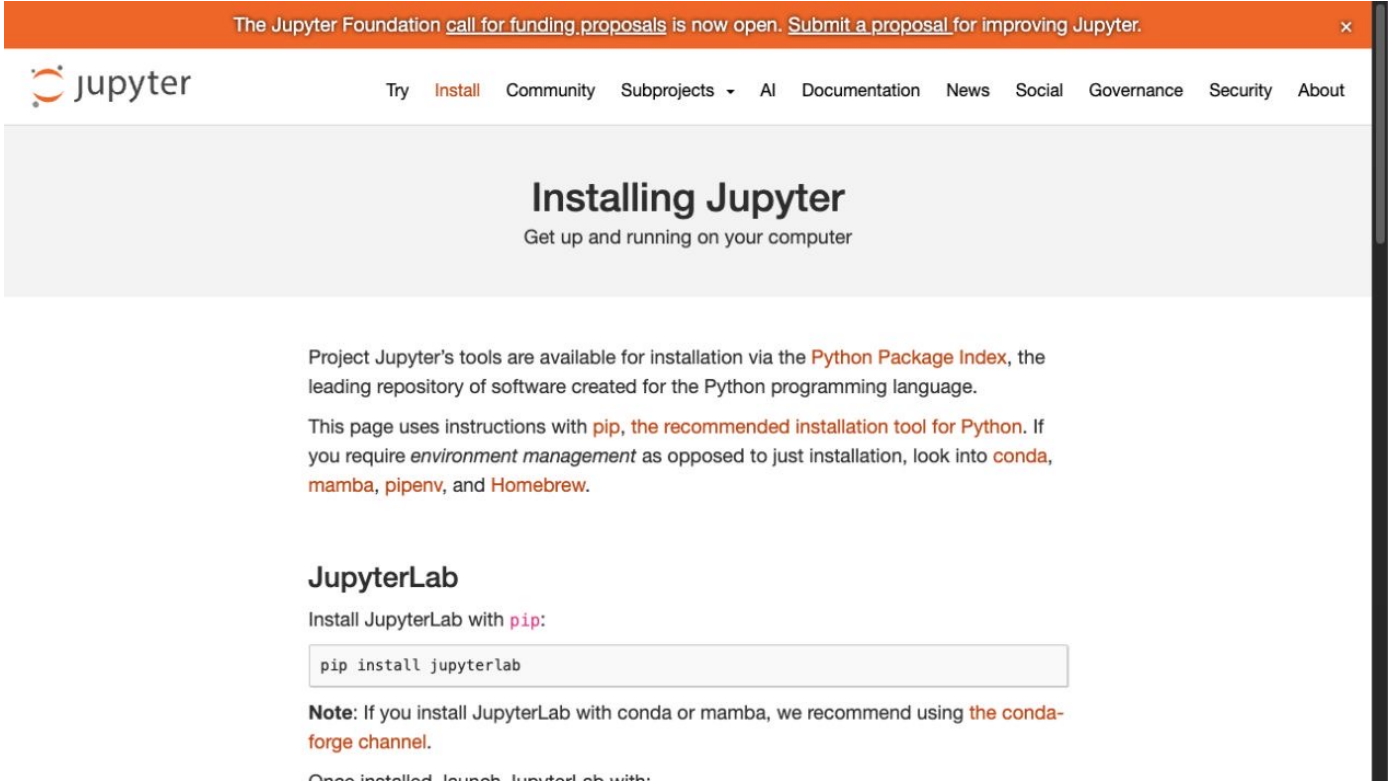


VS Code Python Tutorial 공식 화면

Jupyter 설치 · notebook 실행기

이 화면에서 딱 볼 것

- 현재 kernel에 설치
- Restart Kernel
- Run All
- 실행 결과 저장



The Jupyter Foundation [call for funding proposals](#) is now open. [Submit a proposal](#) for improving Jupyter.

jupyter Try Install Community Subprojects AI Documentation News Social Governance Security About

Installing Jupyter

Get up and running on your computer

Project Jupyter's tools are available for installation via the [Python Package Index](#), the leading repository of software created for the Python programming language.

This page uses instructions with [pip](#), the recommended installation tool for Python. If you require *environment management* as opposed to just installation, look into [conda](#), [mamba](#), [pipenv](#), and [Homebrew](#).

JupyterLab

Install JupyterLab with [pip](#):

```
pip install jupyterlab
```

Note: If you install JupyterLab with conda or mamba, we recommend using the [conda-forge channel](#).

Once installed, launch JupyterLab with:

Jupyter Install 공식 화면

환경 확인 네 줄

코드 전체보다 경계 한 줄을 먼저 봅니다

- 명령과 기대 결과
- PATH 문제 확인
- 새 terminal 재시작
- 캡처에 개인정보 최소화

```
python3 --version  
git --version  
code --version  
python3 -m pip --version
```

가상환경 명령 · macOS/Linux

코드 전체보다 경계 한 줄을 먼저 봅니다

- .venv 생성
- 활성화
- dependency 설치
- interpreter 경로 확인

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-day1.txt
python -c "import sys; print(sys.executable)"
```

가상환경 명령 · Windows PowerShell

코드 전체보다 경계 한 줄을 먼저 봅니다

- ExecutionPolicy 오류 분리
- py launcher
- 같은 requirements
- 경로 화면 확인

```
py -3 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements-day1.txt
python -c "import sys; print(sys.executable)"
```

작은 branch에서 Draft PR까지

코드 전체보다 경계 한 줄을 먼저 봅니다

- 교육용 repo만
- 관련 파일만 stage
- test·diff 증거
- gh가 없으면 GitHub 화면

```
git switch -c test/reject-python-file
python3 -m pytest -q
git diff --check
git add tests/test_day1_agent.py
git commit -m "test: reject python file reads"
git push -u origin HEAD
gh pr create --draft --fill
```

AGENTS.md · Codex가 볼 Review 기준

코드 전체보다 경계 한 줄을 먼저 봅니다

- 저장소 특화 기준
- 결과 중심 규칙
- 기계 검사는 CI
- 최종 판단은 사람

Code Review Rules

Safety boundary

- workspace 밖 파일 접근을 허용하지 않는다.
- 외부 쓰기는 사람 승인 없이 실행하지 않는다.

Tests

- 동작 변경에는 정상·실패 unit test를 함께 둔다.

개발환경 세팅 실습 1 · 04 notebook 열기

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 저장소를 VS Code로 엽니다.
- 2 04 notebook을 엽니다.
- 3 현재 .venv kernel을 고릅니다.
- 4 repo_ready=True를 확인합니다.

완료 증거 | 같은 Python·repo 확인

개발환경 세팅 실습 2 · library 설치 셀

10 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 상단 설치 셀을 실행합니다.
- 2 필수와 선택 결과를 구분합니다.
- 3 설치가 있었다면 Kernel을 재시작합니다.
- 4 첫 셀부터 다시 실행합니다.

완료 증거 | ALREADY_READY 또는 INSTALLED

소프트웨어 실습 1 · Ollama 상태 진단

10 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 probe_ollama 셀을 실행합니다.
- 2 CLI·server·model 값을 읽습니다.
- 3 error_code를 확인합니다.
- 4 recommended_lane을 기록합니다.

완료 증거 | Ollama 상태 JSON

소프트웨어 실습 2 · 좁은 test 실행

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 test_ollama_tool_agent.py만 실행합니다.
- 2 4 passed를 확인합니다.
- 3 실패하면 첫 오류부터 봅니다.
- 4 실행 화면을 저장합니다.

완료 증거 | 4 passed

실패 1 · 설치를 끝없이 붙잡는다

현업에서 자주 깨지는 방식

- 한 명의 PATH에 30분
- 전체 대기
- 학습 목표 상실
- 다운로드 경쟁

제가 먼저 고치는 순서

- 10분 timebox
- checkpoint zip
- fixture lane
- 점심 후 재합류

실패 2 · 전역 Python에 모두 설치

현업에서 자주 깨지는 방식

- 버전 충돌
- 권한 오류
- 재현 불가
- 다른 프로젝트 영향

제가 먼저 고치는 순서

- .venv
- python -m pip
- requirements 고정
- interpreter 확인

실패 3 · Agent에게 저장소 전체 수정 허용

현업에서 자주 깨지는 방식

- 범위 모호
- 불필요 파일 변경
- test 삭제
- secret 접근

제가 먼저 고치는 순서

- 허용 파일
- 금지 조건
- diff 우선
- test 결과 요구

재직자 · 회사 PC 제약 lane

제약

- 관리자 권한 없음
- 외부 다운로드 차단
- GitHub 접속 제한
- GPU 없음

대체

- portable/승인 절차
- 사전 제공 fixture
- 로컬 Git
- 작은 CPU 모델·텍스트

Gate 4 · 5차시 Tool Call을 시작할 수 있습니까?

상황: Ollama 앱과 server는 실행되지만 model 목록이 비어 있습니다.

1 probe 결과에서 준비된 것과 빠진 것은 무엇일까?

2 수업 중 다운로드 대신 어떤 lane을 선택할까?

3 4 passed와 상태 JSON을 어디에 남길까?

12:50-13:40

1일차 5차시

Ollama Tool Calling과 안전 실행

12:50-13:00 강의 · 13:00-13:10 코드 시연 · 13:10-13:35 소프트웨어 실습 · 13:35-13:40 확인

지금 배우는 내용	이 차시에서 이해할 것	함께 챙길 내용
Qwen · 검증 · 실행 · Test	모델은 제안하고 Python 코드가 실행을 허가한다	정상 · 차단 · provider

LLM을 붙이기 전에 실패 상태부터 고정합니다

이 실행 루프는 언제 재시도하고, 언제 사람에게 넘기고, 언제 완전히 멈추는가?

계획

규칙 기반 planner로 next action 선택

검사

validator가 schema와 policy 확인

실행

executor가 좁은 tool만 호출

관찰

ToolResult로 성공·실패를 정규화

중단

timeout·attempt·terminal status로 종료

Agent 실행 루프를 나누는 네 역할

용어	쉽게 말하면	오늘 쓰는 곳
Planner Next-action Selector	현재 요청을 보고 다음에 사용할 도구와 인자를 정하는 부분이다.	‘txt를 읽어줘’에서 read_public_text와 path를 추출한다.
Executor Validated Runner	Planner의 요청을 다시 검사한 뒤 실제 도구 함수를 실행하는 부분이다.	Registry에 있는 허용 도구만 호출한다.
Timeout Maximum Wait	도구가 응답하지 않을 때 기다릴 최대 시간이다.	10초를 넘으면 timeout_error로 기록하고 fallback을 선택한다.
Idempotency Same Request, One Effect	같은 요청을 여러 번 실행해도 외부 결과가 한 번만 생기게 하는 성질이다.	request_id가 이미 성공했으면 저장된 결과를 반환한다.

Fixture와 Qwen은 같은 Tool 계약을 쓴다

먼저 결정론적 fixture로 검증한 뒤 provider만 Ollama로 바꿔 모델 변동과 코드 오류를 분리한다.

- same tool schema
- provider requested·used
- fallback reason

운영에서 먼저 터지는 곳 | 무한 retry는 회복력이 아니라 장애를 늦게 발견하는 방식이다.

Planner는 “무엇을 할까”만 결정

도구 이름과 인자를 제안하고 실행 권한은 갖지 않는다.

- input parse
- planned_call
- no side effect

Validator는 안전 경계를 고정

허용 도구·필수 인자·타입·추가 필드·경로를 검사한다.

- schema
- policy
- path guard
- error code

Executor는 등록 함수만 호출

동적 eval이나 임의 import 없이 registry에서 함수 객체를 찾는다.

- registry lookup
- kwargs
- exception boundary

운영에서 먼저 터지는 곳 | 결정론적 planner로 통과하지 못한 계약은 LLM을 넣어도 안정되지 않는다.

ToolResult는 성공과 실패의 공통 언어

downstream graph가 traceback 구조를 몰라도 다음 행동을 고를 수 있다.

- ok
- data
- error_code
- message

read_public_text는 의도적으로 좁다

workspace 안의 공개 txt·md만 읽는 도구로 최소 권한을 체험한다.

- root boundary
- suffix allowlist
- UTF-8

Provider 요청과 실제 사용을 따로 남긴다

Qwen 요청이 실패해 fixture로 복구돼도 성공처럼 숨기지 않는다

-
- provider_requested
- provider_used
- fallback_reason

운영에서 먼저 터지는 곳 | Agent의 실력은 성공 횟수보다 실패 상태를 얼마나 잘 구분하는지에서 드러난다.

코드 읽기 순서

01

DATA

dataclass·schema

02

POLICY

validate·guard

03

EXECUTE

registry·result

04

DEMO

__main__

Run All 순서

01

KERNEL

interpreter

02

IMPORT

module

03

NORMAL

success

04

FAIL

4 error cases

실패 주입 순서

01	02	03	04
UNKNOWN	MISSING	TRAVERSAL	DUPLICATE
도구 없음	인자 누락	경로 이탈	cache 확인

pytest 빨강→초록

01

RUN

현재 실패

02

READ

첫 오류

03

FIX

최소 변경

04

RERUN

전체 25개

Git checkpoint 만들기

01

STATUS

변경 분리

02

DIFF

의도 확인

03

STAGE

src·tests만

04

COMMIT

메시지·log

이 네 화면은 실행보다 복구 경로를 확인하기 위한 것입니다

Notebook 실행기 → interpreter → Agent 수정 → 같은 repo 검토

01	———— Notebook kernel을 준비	Run All 가능
02	———— Terminal과 Notebook Python 맞추기	같은 interpreter
03	———— 작은 spec으로 한 경계만 수정	제한된 diff
04	———— 다른 Agent로 실패 조건 검토	test 재실행

실제 notebook · Qwen Tool Call 또는 fallback

이 화면에서 딱 볼 것

- provider_requested=ollama
- provider_used 확인
- fallback_reason
- automatic_email=false

5차시 · Ollama 실제 Tool Call 제안

아래 셀은 항상 provider="ollama" 를 요청합니다. 감사 컴퓨터처럼 qwen3:4b 가 준비되어 있으면 실제 응답을 사용하고, 수강생 컴퓨터의 모델이 준비되지 않았다면 fixture로 복구한 이유가 결과에 남습니다.

```
In [7]: MODEL = "qwen3:4b"
ollama_tool_result = run_tool_agent(
    "data/meeting_sample_ko.txt를 읽어줘",
    workspace=ROOT,
    provider="ollama",
    model=MODEL,
    allow_fallback=True,
)
print(json.dumps({
    "provider_requested": ollama_tool_result["provider_requested"],
    "provider_used": ollama_tool_result["provider_used"],
    "fallback_reason": ollama_tool_result["fallback_reason"],
    "tool_call": ollama_tool_result["tool_call"],
    "tool_status": ollama_tool_result["status"],
    "automatic_email": ollama_tool_result["automatic_email"],
}, ensure_ascii=False, indent=2))

{
  "provider_requested": "ollama",
  "provider_used": "ollama",
  "fallback_reason": null,
  "tool_call": {
    "name": "read_public_text",
    "arguments": {
      "path": "data/meeting_sample_ko.txt"
    }
  },
  "tool_status": "SUCCESS",
  "automatic_email": false
}

In [8]: OUTPUT = ROOT / "output/notebook-ollama"
OUTPUT.mkdir(parents=True, exist_ok=True)
(OUTPUT / "ollama_tool_result.json").write_text(
    json.dumps(ollama_tool_result, ensure_ascii=False, indent=2),
    encoding="utf-8",
)
```

04 notebook Tool Calling 실행 화면

점심 전 Gate · Agent부터 배포 ZIP까지 통과

이 화면에서 딱 볼 것

- 25 passed
- [100%]
- Agent·Chain·Graph·STT
- 배포 bundle test 포함

5차시 · 점심 전 전체 회귀 test

새로 만든 Ollama 경로뿐 아니라 기존 Agent·LangChain·LangGraph·STT 계약이 함께 유지되는지 전체 Day 1 test를 실행합니다. 실패하면 마지막 줄보다 첫 번째 오류부터 복구합니다.

```
In [9]: full_test = subprocess.run(
    [sys.executable, "-m", "pytest", "-q"],
    cwd=ROOT,
    capture_output=True,
    text=True,
    check=False,
)
print(full_test.stdout.strip())
assert full_test.returncode == 0, full_test.stderr

.....
25 passed in 0.33s [100%]
```

6차시 · LangChain LCEL Provider 교체

같은 transcript와 Pydantic schema를 유지하고 provider만 바꿉니다.

Prompt → fixture/Ollama → Pydantic Parser → Policy Validator

Ollama용 선택 패키지는 다음과 같이 설치합니다.

python -m pip install -r requirements-local-llm-optional.txt

```
In [10]: from src.langchain_lab import run_langchain_lab

transcript = (ROOT / "data/meeting_sample_ko_12min.txt").read_text(encoding="utf-8")
fixture_chain = run_langchain_lab(transcript, provider="fixture")
print(json.dumps({
    "provider_used": fixture_chain["provider_used"],
    "pipeline": fixture_chain["pipeline"],
    "checks": fixture_chain["checks"],
}, ensure_ascii=False, indent=2))

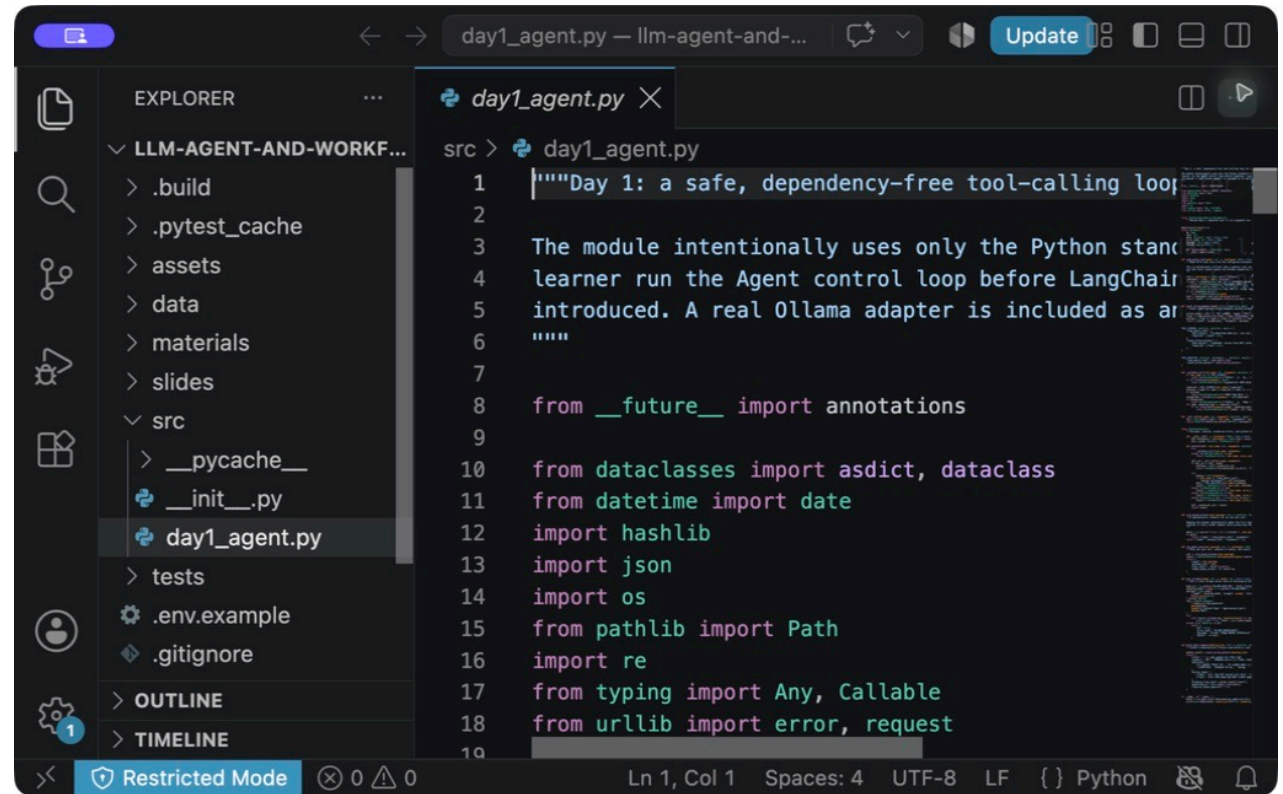
{
  "provider_used": "fixture",
  "pipeline": [
    "OllamaToolCall"
```

Agent·Chain·Graph·STT·bundle 회귀 test

실제 Agent 코드 · 실행 권한은 Python에

이 화면에서 딱 볼 것

- allowlist
- workspace 경계
- ToolResult
- 정규화된 오류



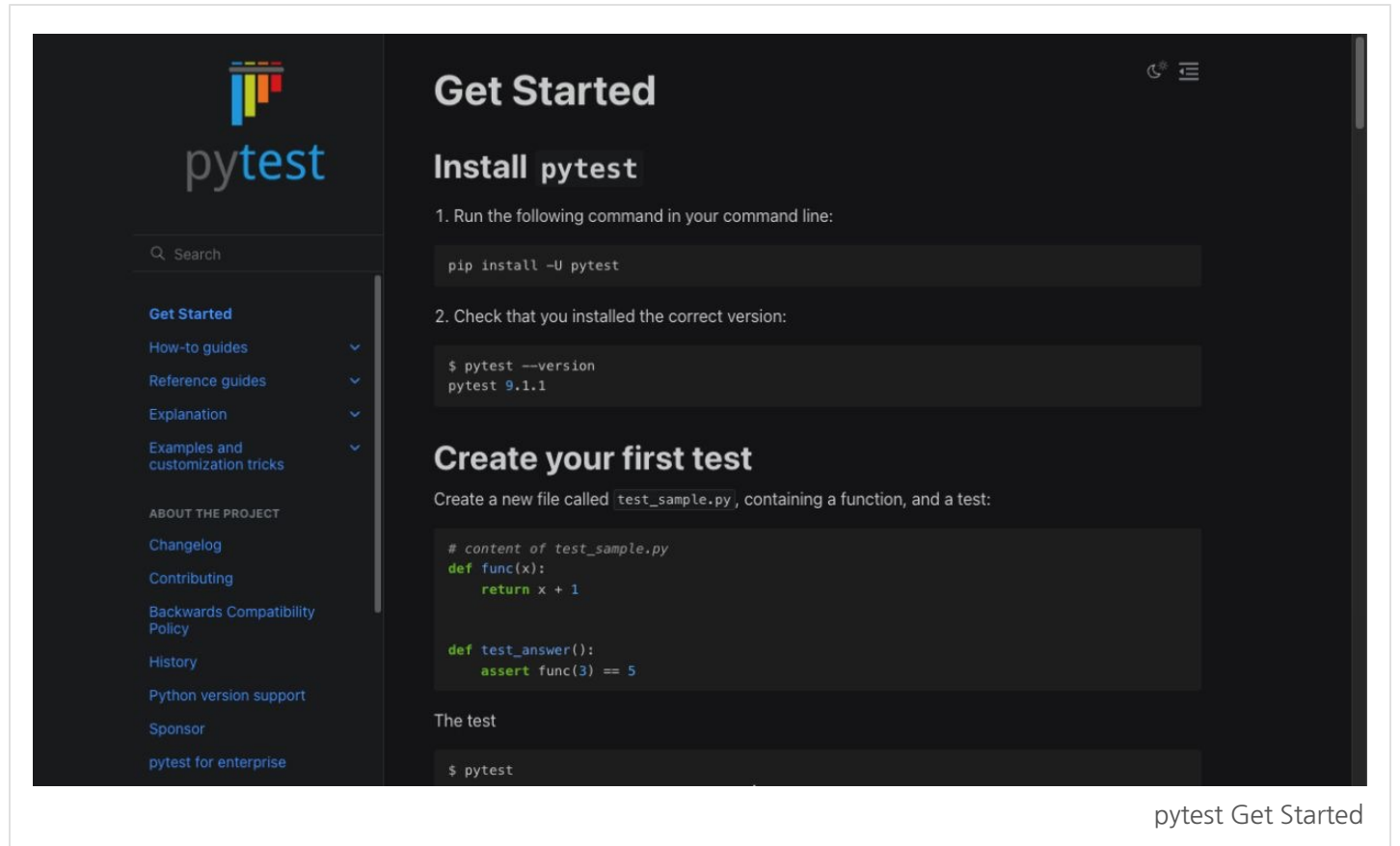
```
src > day1_agent.py
1  """Day 1: a safe, dependency-free tool-calling loop
2
3  The module intentionally uses only the Python standard library to
4  learner run the Agent control loop before LangChain is
5  introduced. A real Ollama adapter is included as an example.
6  """
7
8  from __future__ import annotations
9
10 from dataclasses import asdict, dataclass
11 from datetime import date
12 import hashlib
13 import json
14 import os
15 from pathlib import Path
16 import re
17 from typing import Any, Callable
18 from urllib import error, request
19
```

VS Code의 SafeToolExecutor

pytest · 실패를 재현하는 가장 작은 도구

이 화면에서 딱 볼 것

- 정상·공격 함께
- assert 계약
- 좁은 test 먼저
- 전체 회귀 test



규칙 기반 planner

코드 전체보다 경계 한 줄을 먼저 봅니다

- 결정론적 plan
- 파일명만 추출
- unknown도 정상 fixture
- Day2 model로 교체

```
def rule_based_planner(message):  
    match = re.search(r"([\\w.\\-]+\\.txt|md)", message)  
    if match:  
        return {"name": "read_public_text",  
                "arguments": {"path": match.group(1)}}  
    return {"name": "unknown_tool", "arguments": {}}
```

SafeToolExecutor의 경계

코드 전체보다 경계 한 줄을 먼저 봅니다

- 검증 먼저
- registry 함수
- 예외 정규화
- 반환 계약

```
def execute(self, name, args):
    try:
        validate(name, args)
        fn = TOOL_REGISTRY[name]
        data = fn(**args)
        return ToolResult(True, name, data=data)
    except ToolValidationError as exc:
        return ToolResult(False, name,
                           error_code="VALIDATION_ERROR")
```

파일 확장자 allowlist

코드 전체보다 경계 한 줄을 먼저 봅니다

- 목적에 필요한 형식만
- binary 제외
- 정책 메시지
- POLICY_BLOCKED로 매핑

```
if candidate.suffix.lower() not in {".txt", ".md"}:  
    raise ToolValidationError(  
        "Day 1에서는 .txt와 .md만 허용합니다."  
    )
```

중복 호출 cache

코드 전체보다 경계 한 줄을 먼저 봅니다

- 실행 전 조회
- 성공·실패 정책 결정
- cached 표시
- 외부 쓰기에는 영속 저장

```
call_id = make_call_id(name, args)
if call_id in self._cache:
    prev = self._cache[call_id]
    return replace(prev, cached=True)
result = call_tool()
self._cache[call_id] = result
```

Agent event 한 건

코드 전체보다 경계 한 줄을 먼저 봅니다

- 입력
- 결정
- 결과
- 사람 검토 여부

```
event = {  
    "input": user_message,  
    "planned_call": call,  
    "tool_result": result.to_dict(),  
    "needs_human_review": not result.ok  
}
```

소프트웨어 실습 1 · fixture 정상 Tool Call

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 같은 04 notebook을 이어서 씁니다.
- 2 meeting_sample_ko.txt를 요청합니다.
- 3 planned tool과 arguments를 봅니다.
- 4 SUCCESS를 확인합니다.

완료 증거 | 정상 Tool Call JSON

소프트웨어 실습 2 · 경로 이탈 차단

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 ../../private.txt를 입력합니다.
- 2 모델 제안 뒤 validator를 봅니다.
- 3 POLICY_BLOCKED를 확인합니다.
- 4 사람 검토 flag를 확인합니다.

완료 증거 | 차단 ToolResult

소프트웨어 실습 3 · Qwen Tool Call 요청

10 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 provider=ollama 셀을 실행합니다.
- 2 requested와 used를 비교합니다.
- 3 모델이 없으면 fallback 이유를 읽습니다.
- 4 결과 JSON을 저장합니다.

완료 증거 | 실제 또는 fallback JSON

소프트웨어 실습 4 · pytest 전체 확인

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 전체 pytest -q를 실행합니다.
- 2 실패 시 첫 오류만 읽습니다.
- 3 25 passed를 확인합니다.
- 4 화면을 저장합니다.

완료 증거 | 25 passed

실패 1 · Notebook과 module이 다른 Python

현업에서 자주 깨지는 방식

- import error
- 설치했는데 못 찾음
- terminal은 성공
- kernel은 실패

제가 먼저 고치는 순서

- sys.executable 비교
- interpreter 재선택
- kernel restart
- 같은 .venv

실패 2 · 첫 실패 뒤 계속 실행

현업에서 자주 깨지는 방식

- 연쇄 오류
- 원인 혼동
- output 오염
- 시간 낭비

제가 먼저 고치는 순서

- 첫 traceback
- 가장 위 원인
- 최소 셀 재실행
- Restart & Run All

실패 3 · 테스트를 통과시키려고 정책 완화

현업에서 자주 깨지는 방식

- allowlist 제거
- assert 삭제
- broad except
- expected 변경

제가 먼저 고치는 순서

- 요구사항 재확인
- 최소 수정
- 새 regression test
- 정책 유지

재직자 · Safe loop를 현업 tool에 매핑

교육 도구

- txt 읽기
- marker 세기
- 결과 JSON
- 로컬 cache

현업 예시

- FAQ 조회
- 티켓 분류
- 검토 draft
- request dedupe

Gate 5 · 14:00 점심 전 실행 증거

지금부터 점심시간입니다. 14:55까지 돌아오면 같은 notebook의 LangChain LCEL 구간부터 시작합니다.

1

정상과 POLICY_BLOCKED 결과가 남았는가?

2

Qwen 요청의 provider_used와 fallback_reason을 설명할 수 있는가?

3

pytest 25개 통과 화면이 남았는가?

15:00-15:40

1일차 6차시

LangChain LCEL과 Qwen Provider

15:00-15:08 개념 · 15:08-15:18 fixture 시연 · 15:18-15:35 provider 교체 실습 · 15:35-15:40 확인

지금 배우는 내용	이 차시에서 이해할 것	함께 챙길 내용
Prompt · Provider · Parser · Policy	provider가 바뀌어도 typed output과 정책은 유지된다	typed JSON · provider

프로그램이 달라도 generate 계약은 하나로 둡니다

특정 모델과 프로그램이 사라져도 이 실습의 핵심 계약은 그대로 실행되는가?

선택

PC 사양과 UI 선호로 provider 결정

연결

health와 base URL을 확인

변환

adapter가 요청·응답 차이를 흡수

검증

JSON parse와 schema를 순서대로 확인

대체

연결 실패 시 fixture로 계속 진행

모델을 교체하기 위한 네 가지 층

용어	쉽게 말하면	오늘 쓰는 곳
Local LLM Model on My Computer	외부 API가 아니라 내 PC에서 실행되는 언어 모델이다.	Ollama에서 qwen3 계열 작은 모델을 실행한다.
Provider Model Access Method	OpenAI·Anthropic·Ollama처럼 모델을 호출하는 서비스 또는 실행 방식이다.	local provider와 fixture provider를 같은 인터페이스로 부릅니다.
Adapter Common Interface	서로 다른 Provider 호출 방식을 하나의 공통 함수로 감싸는 계층이다.	generate(prompt) 함수 뒤에 Ollama·LM Studio·fixture를 숨깁니다.
Fallback Alternative Path	주 경로가 실패했을 때 학습이나 업무를 계속할 대체 경로이다.	Local LLM이 없으면 고정 JSON fixture로 Schema·Graph 실습을 계속한다.

LangChain은 네 단계를 실행 순서로 묶는다

Prompt·Provider·Parser·Validator가 LCEL 파이프로 이어지고 각 단계는 따로 바꿀 수 있다.

- ChatPromptTemplate | Model Adapter | PydanticOutputParser | Policy Validator

무료 소프트웨어 실습의 조건 | 무료는 비용 0원이 아니라 선택권이 남아 있는 구조다.

Ollama는 CLI와 localhost API 경로

모델 pull·serve·generate를 간단한 로컬 흐름으로 제공한다.

- ollama pull
- localhost:11434
- API key 없음

Fixture는 수업용 가짜 답이 아니라 test double

항상 같은 typed 결과를 만들어 Prompt 이후 단계와 정책을 결정론적으로 검증한다.

- same transcript
- stable JSON
- regression test

Pydantic Parser가 자연어를 업무 타입으로 바꾼다

필드·날짜·근거 배열을 검증해 그럴듯한 문장을 제품 데이터와 구분한다.

- MeetingBrief
- ActionItem
- evidence_ids

무료 소프트웨어 실습의 조건 | adapter가 없으면 모델 교체가 곧 프로젝트 재개발이 된다.

Provider adapter가 차이를 감춘다

URL·인증·payload·response를 provider 내부로 모으고 앱에는 공통 result를 준다.

- generate(prompt)
- success data
- normalized error

OpenAI-compatible은 완전 동일을 뜻하지 않는다

model name·지원 필드·tool calling·structured output 차이를 확인한다.

- endpoint
- payload subset
- model capability

작은 모델은 출력 계약을 더 엄격히

JSON parse·schema validate·repair·human review를 별도 단계로 둔다.

- extract JSON
- validate
- bounded repair
- stop

무료 소프트웨어 실습의 조건 | local LLM은 정답이 아니라 데이터·비용·지연시간 사이의 선택지다.

LCEL 네 단계

01

PROMPT

회의·규칙

02

PROVIDER

fixture·Qwen

03

PARSE

Pydantic

04

POLICY

메일·승인

같은 notebook 실행

01

INPUT

같은 transcript

02

FIXTURE

기준 실행

03

OLLAMA

provider 교체

04

COMPARE

requested·used

Provider adapter

01

MAP

provider payload

02

CALL

timeout

03

NORMALIZE

공통 result

04

FALLBACK

이유 기록

Structured output 처리

01

RAW

원문 보존

02

PARSE

JSON 추출

03

VALIDATE

schema

04

REVIEW

repair·human

모델 오류 분류

01

CONNECT

서버 없음

02

MODEL

tag 없음

03

FORMAT

JSON 깨짐

04

QUALITY

근거 누락

세 프로그램의 차이는 화면에서, 공통점은 adapter에서 봅니다

Ollama → model tag → LM Studio server → Jan API server

도구·서비스	도메인·역할	이번 강의에서 쓰는 이유	유사 도구와의 차이
Ollama	CLI·localhost API	명령 몇 줄로 local model과 API를 함께 실행	GUI보다 단순하고 자동화에 유리
LM Studio	GUI·Local Server	모델 탐색과 server 상태를 화면으로 확인	초심자 친화적이나 GUI 조작이 필요
Jan	Desktop Local API	OpenAI-compatible 대체 경로를 하나 더 확보	생태계·문서 범위는 도구별 확인
Fixture Provider	설치 없는 고정 응답	모델 설치가 실패해도 Schema·Graph 실행 지속	생성 품질은 평가할 수 없는 복귀 경로

실제 notebook · LangChain LCEL typed output

이 화면에서 딱 볼 것

- 네 단계 pipeline
- provider_used
- schema_valid
- automatic_email_blocked

6차시 · LangChain LCEL Provider 교체

같은 transcript와 Pydantic schema를 유지하고 provider만 바꿉니다.

Prompt → fixture/Ollama → Pydantic Parser → Policy Validator

Ollama용 선택 패키지는 다음과 같이 설치합니다.

python -m pip install -r requirements-local-llm-optional.txt

```
In [10]: from src.langchain_lab import run_langchain_lab

transcript = (ROOT / "data/meeting_sample_ko_12min.txt").read_text(encoding="utf-8")
fixture_chain = run_langchain_lab(transcript, provider="fixture")
print(json.dumps({
    "provider_used": fixture_chain["provider_used"],
    "pipeline": fixture_chain["pipeline"],
    "checks": fixture_chain["checks"],
}, ensure_ascii=False, indent=2))

{
  "provider_used": "fixture",
  "pipeline": [
    "ChatPromptTemplate",
    "Model Adapter",
    "PydanticOutputParser",
    "Policy Validator"
  ],
  "checks": {
    "schema_valid": true,
    "evidence_present": true,
    "automatic_email_blocked": true,
    "human_approval_required": true
  }
}

In [11]: ollama_chain = run_langchain_lab(
    transcript,
    provider="ollama",
    model=MODEL,
    allow_fallback=True,
```

04 notebook LCEL 실행 화면

실제 notebook · Qwen Tool Call 연결 결과

이 화면에서 딱 볼 것

- Qwen 요청
- 실제 사용 provider
- fallback reason
- 같은 Tool 계약

5차시 · Ollama 실제 Tool Call 제안

아래 셀은 항상 `provider="ollama"` 를 요청합니다. 감사 컴퓨터처럼 `qwen3:4b` 가 준비되어 있으면 실제 응답을 사용하고, 수강생 컴퓨터의 모델이 준비되지 않았다면 fixture로 복구한 이유가 결과에 남습니다.

```
In [7]: MODEL = "qwen3:4b"
ollama_tool_result = run_tool_agent(
    "data/meeting_sample_ko.txt를 읽어줘",
    workspace=ROOT,
    provider="ollama",
    model=MODEL,
    allow_fallback=True,
)
print(json.dumps({
    "provider_requested": ollama_tool_result["provider_requested"],
    "provider_used": ollama_tool_result["provider_used"],
    "fallback_reason": ollama_tool_result["fallback_reason"],
    "tool_call": ollama_tool_result["tool_call"],
    "tool_status": ollama_tool_result["status"],
    "automatic_email": ollama_tool_result["automatic_email"],
}, ensure_ascii=False, indent=2))

{
  "provider_requested": "ollama",
  "provider_used": "ollama",
  "fallback_reason": null,
  "tool_call": {
    "name": "read_public_text",
    "arguments": {
      "path": "data/meeting_sample_ko.txt"
    }
  },
  "tool_status": "SUCCESS",
  "automatic_email": false
}

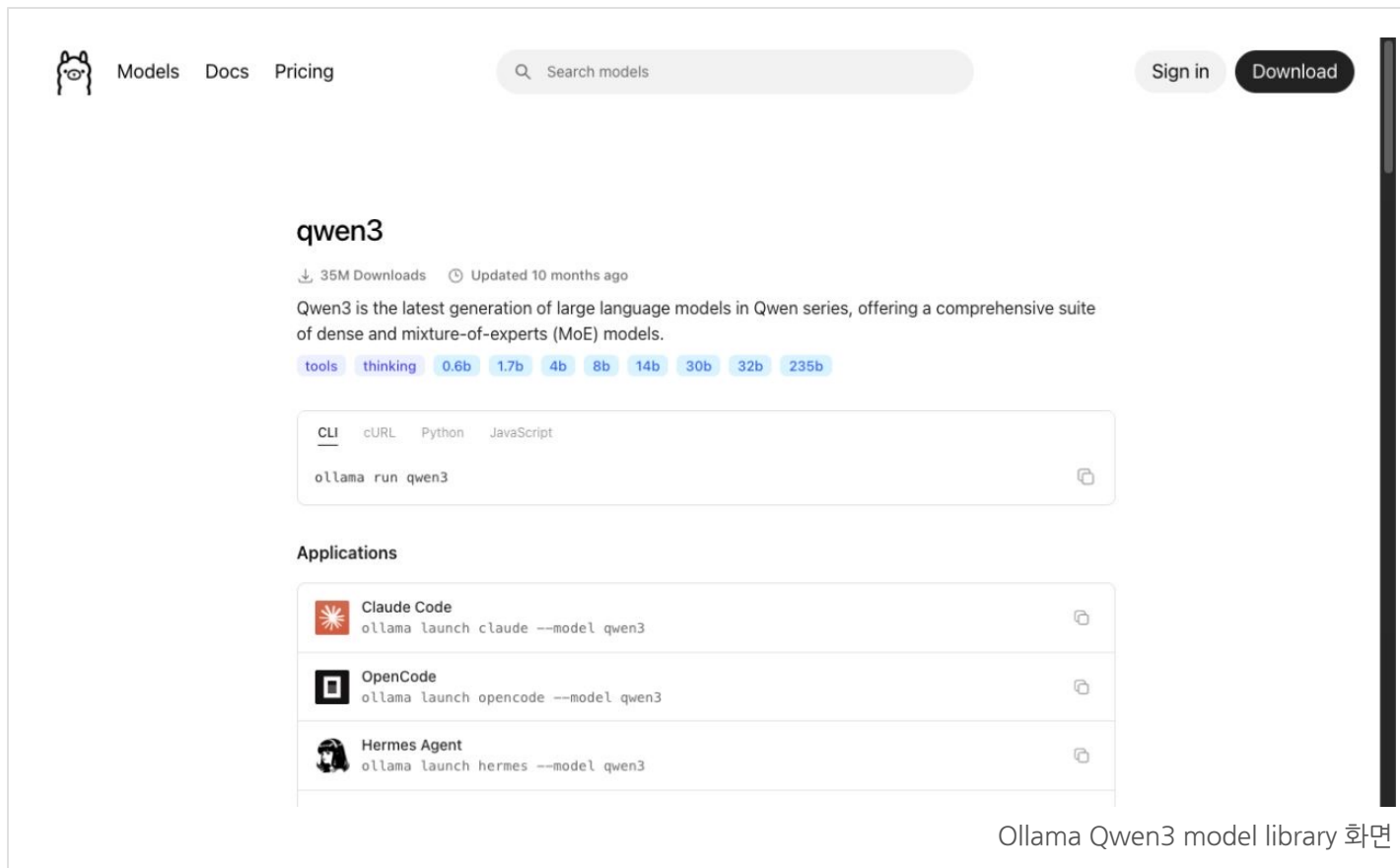
In [8]: OUTPUT = ROOT / "output/notebook-ollama"
OUTPUT.mkdir(parents=True, exist_ok=True)
(OUTPUT / "ollama_tool_result.json").write_text(
    json.dumps(ollama_tool_result, ensure_ascii=False, indent=2),
    encoding="utf-8",
)
```

04 notebook Qwen 요청 화면

Qwen3 model library · tag 확인

이 화면에서 딱 볼 것

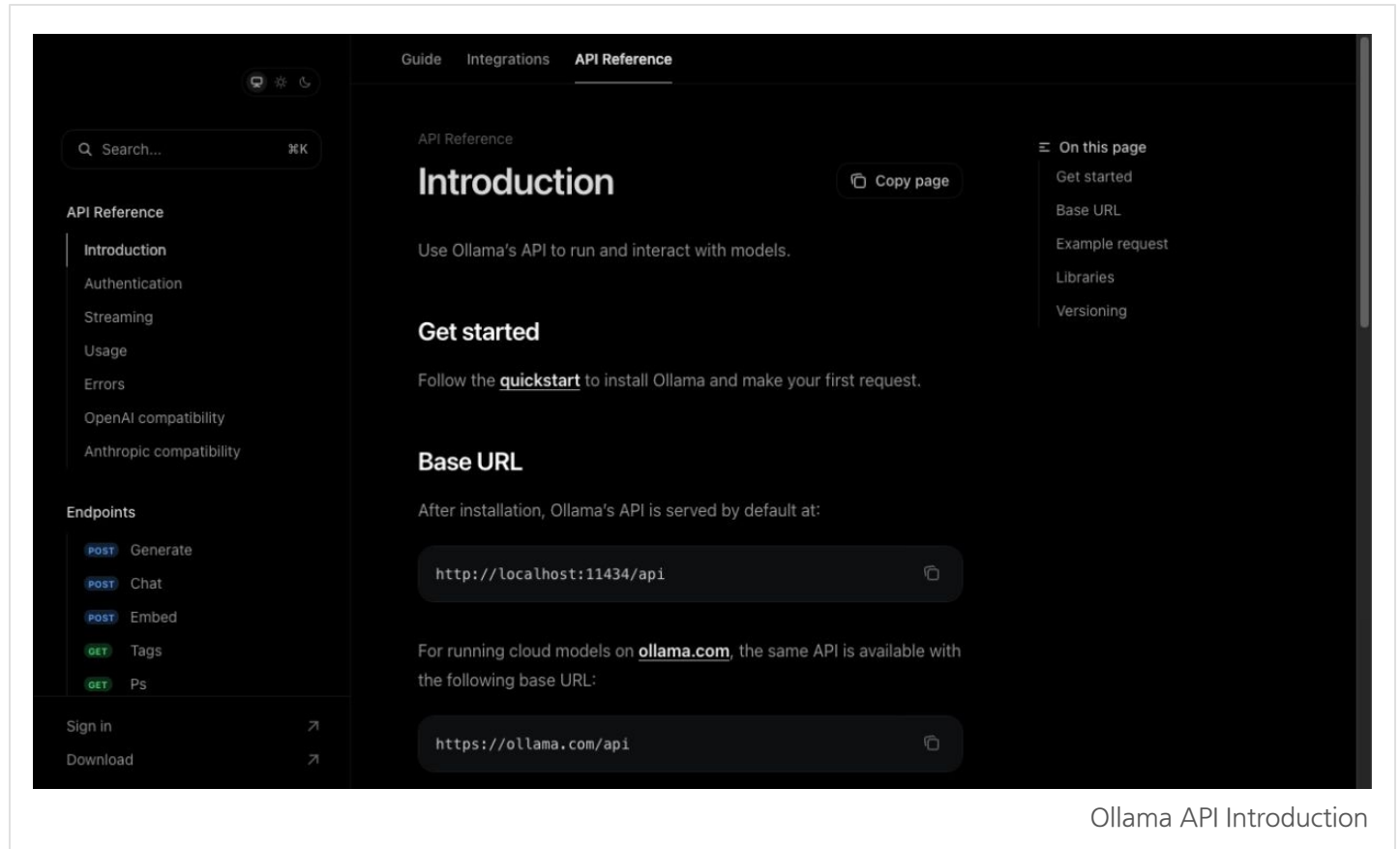
- qwen3:4b
- 약 2.5GB
- 수업 전 설치
- ollama list 확인



Ollama API · localhost 호출

이 화면에서 딱 볼 것

- localhost:11434
- API key 없음
- timeout
- adapter 뒤에 배치



Ollama generate payload

코드 전체보다 경계 한 줄을 먼저 봅니다

- localhost
- model tag
- stream 단순화
- timeout 필수

```
payload = {  
  "model": "qwen3:4b",  
  "prompt": prompt,  
  "stream": False  
}  
POST http://localhost:11434/api/generate
```

연결 실패도 result

코드 전체보다 경계 한 줄을 먼저 봅니다

- exception 밖으로 누출 금지
- 안정된 코드
- 상세 원인
- fallback 선택

```
try:
    data = post_json(url, payload, timeout=60)
    return {"ok": True, "data": data}
except URLERror as exc:
    return {"ok": False,
            "error_code": "OLLAMA_UNAVAILABLE",
            "detail": str(exc)}
```

공통 Provider protocol

코드 전체보다 경계 한 줄을 먼저 봅니다

- 앱과 provider 분리
- 같은 입력
- 같은 result
- test double

```
class Provider(Protocol):  
    def generate(self, prompt: str) -> ModelResult: ...  
  
providers = {  
    "fixture": FixtureProvider(),  
    "ollama": OllamaProvider()  
}
```

JSON parse와 schema validate

코드 전체보다 경계 한 줄을 먼저 봅니다

- raw 보존
- parse 단계
- typed validate
- 근거 정책

```
raw = provider.generate(prompt)
parsed = extract_json(raw.text)
result = MeetingResult.model_validate(parsed)
if missing_evidence(result):
    return NEEDS_REVIEW
```


bounded repair · 한 번만

코드 전체보다 경계 한 줄을 먼저 봅니다

- 무한 repair 금지
- 오류 정보 제공
- 최대 횟수
- 사람 검토

```
for attempt in range(2):  
    try:  
        return validate(parse(text))  
    except SchemaError as err:  
        if attempt == 1: break  
        text = provider.generate(repair_prompt(err, text))  
return NEEDS_REVIEW
```

소프트웨어 실습 1 · fixture LCEL 실행

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 같은 transcript를 읽습니다.
- 2 fixture chain 셀을 실행합니다.
- 3 pipeline 네 단계를 확인합니다.
- 4 정책 check 네 개를 봅니다.

완료 증거 | typed JSON + checks

소프트웨어 실습 2 · Qwen provider 교체

10 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 provider=ollama로 다시 실행합니다.
- 2 요청 provider를 확인합니다.
- 3 실제 provider와 이유를 봅니다.
- 4 같은 schema를 확인합니다.

완료 증거 | provider 비교 결과

실행 확인 1 · 공통 contract 비교

07 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 fixture 결과와 Qwen 요청 결과를 엽니다.
- 2 공통 필드를 나란히 봅니다.
- 3 provider 전용 필드를 구분합니다.
- 4 다음 Graph에 넘길 필드를 확인합니다.

완료 증거 | provider 비교표

소프트웨어 실습 3 · 정책 실패 확인

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 automatic_email 값을 확인합니다.
- 2 사람 승인 필드를 확인합니다.
- 3 evidence 누락 시 HOLD 기준을 봅니다.
- 4 정책 검사를 결과에 저장합니다.

완료 증거 | policy checks JSON

실패 1 · 큰 모델 다운로드를 기다림

현업에서 자주 깨지는 방식

- 수업 전체 대기
- 디스크 부족
- RAM swap
- 학습 목표 상실

제가 먼저 고치는 순서

- 작은 모델 하나
- 사전 설치
- fixture 즉시 전환
- 모델 비교는 심화

실패 2 · 연결 실패를 traceback으로

현업에서 자주 깨지는 방식

- 노트북 중단
- 학생 원인 파악 어려움
- graph state 유실
- fallback 불가

제가 먼저 고치는 순서

- error_code
- 사용자 메시지
- trace detail
- same result contract

실패 3 · JSON처럼 보이면 통과

현업에서 자주 깨지는 방식

- 필드 누락
- 타입 오류
- 근거 없음
- 날짜 형식 혼합

제가 먼저 고치는 순서

- typed schema
- evidence policy
- date validator
- human review

재직자 · 로컬 LLM 도입 질문

기술 질문

- RAM·속도
- model capability
- API 호환
- 배포 형태

운영 질문

- 데이터 경계
- 업데이트 책임
- 관측·감사
- fallback·SLA

Gate 6 · Local LLM이 꺼져도 계속하기

상황: Ollama 연결이 거부되고, 교실 네트워크에서는 새 모델도 받을 수 없습니다.

1

같은 Provider contract로 어떤 fallback을 연결할까?

2

연결 실패를 traceback 대신 어떤 result로 돌려줄까?

3

fixture로도 끝까지 검증할 출력 계약은 무엇일까?

15:40-16:20

1일차 7차시

LangChain·LangGraph와 사람 승인

15:40-15:48 개념 · 15:48-15:55 LCEL 시연 · 15:55-16:18 소프트웨어 실습 · 16:18-16:20 확인

지금 배우는 내용	이 차시에서 이해할 것	함께 챙길 내용
LCEL · State · Node · Checkpoint · 승인·수정·거절	중요한 행동 전에는 멈추고 사람이 결정한다	승인·수정·거절 상태

사람 검토를 버튼이 아니라 State 전이로 만듭니다

사람이 개입한 뒤에도 같은 요청과 같은 상태에서 안전하게 이어갈 수 있는가?

상태

다음 node에 필요한 필드만 선언

분기

edge에 retry·review·failed 조건을 둬

중단

interrupt payload로 근거와 초안을 전달

결정

approve·edit·reject event를 저장

재개

같은 thread에서 idempotent하게 이어감

중단과 재개를 설명하는 네 단어

용어	쉽게 말하면	오늘 쓰는 곳
State Workflow Memory	Graph의 각 단계가 읽고 갱신하는 구조화된 현재 상태이다.	transcript·draft·errors·approval·status를 필드로 둡니다 .
Node / Edge Step / Route	Node는 한 책임의 작업이고 Edge는 다음 단계로 가는 조건이다.	validate Node가 READY면 review, 오류면 retry 또는 failed로 이동한다.
Checkpoint Saved State	Workflow의 중간 State를 저장해 나중에 같은 지점에서 이어갈 수 있게 한다.	thread_id로 저장된 State를 불러와 승인 뒤 재개한다.
HITL Human in the Loop	중요한 결정이나 외부 변경 전에 사람이 확인하고 승인·수정·거절하는 단계이다.	낮은 confidence Action Item은 reviewer가 근거를 보고 승인한다.

State는 다음 행동의 근거

대화 전체가 아니라 node가 판단에 필요한 최소 구조를 저장한다.

- request_id·step
- draft·errors
- retry·approval·status

Graph가 필요한 이유 | State에 모든 것을 넣는 순간 Graph는 기억장치가 아니라 쓰레기통이 된다.

Node는 한 가지 책임의 함수

입력 state를 읽고 부분 update를 반환하도록 작게 나눈다.

- transcribe
- summarize
- validate
- review

Edge는 다음 node를 정하는 정책

성공·오류·위험·승인 결과에 따라 명시적으로 분기한다.

- conditional route
- END
- retry loop

Checkpoint가 중단과 재개를 가능하게

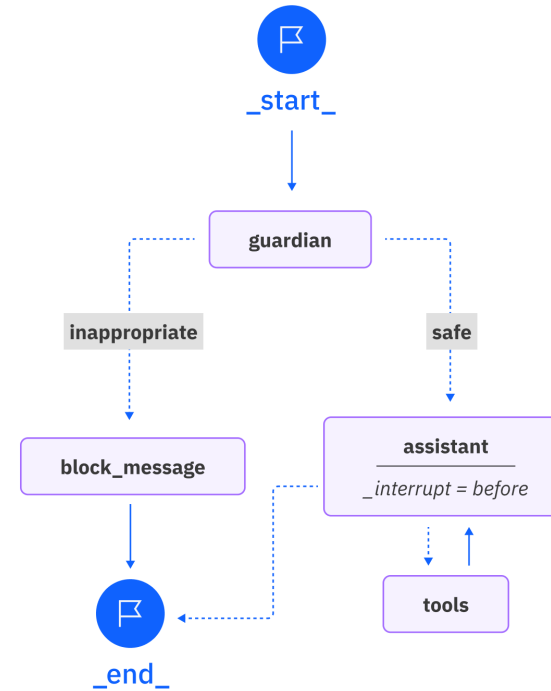
각 step의 state snapshot을 thread 단위로 저장한다.

- checkpoint
- thread_id
- state history

Graph가 필요한 이유 | Human Approval은 버튼이 아니라 근거·결정·책임을 남기는 event다.

Interrupt는 “잠깐 멈춤”이 아니라 업무 상태다

- 어디에서 멈추는가
- 사람이 무엇을 보는가
- 같은 상태로 어떻게 재개하는가



Approve·Edit·Reject는 다른 event

승인은 원안 진행, 수정은 새 payload, 거절은 이유와 종료·재계획을 남긴다.

- decision
- reviewer
- reason
- edited payload

Retry는 일시 오류에만

오류 class·max attempts·backoff·stop을 state에 남긴다.

- transient
- attempts
- next delay
- exhausted

Graph가 필요한 이유 | Idempotency가 없는 resume은 자동화가 아니라 중복 사고의 예약이다.

Graph 설계 순서

01

STATE

필드·status

02

NODES

한 책임

03

EDGES

분기·END

04

CHECK

중단·재개

오류 routing 순서

01

CLASSIFY

오류 class

02

COUNT

attempt

03

ROUTE

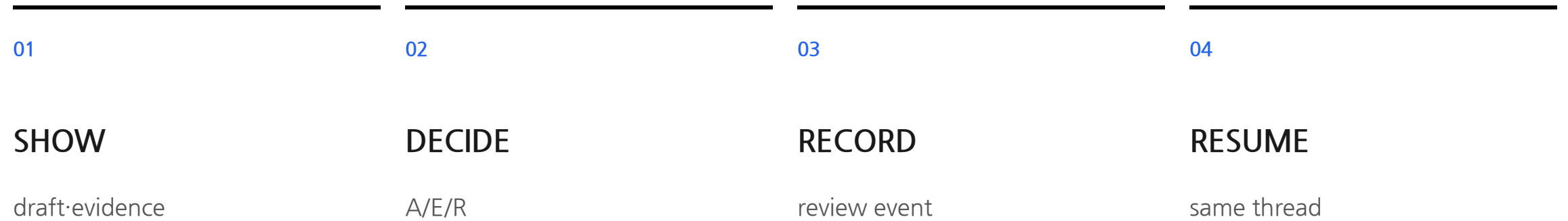
retry·review

04

STOP

terminal status

Human review node



Interrupt payload 설계

01

SUMMARY

무엇을 하려나

02

TARGET

어디에 쓰나

03

RISK

왜 검토하나

04

OPTIONS

승인·수정·거절

Idempotent publish

01

KEY

request_id

02

LOOKUP

기존 성공

03

WRITE

한 번 수행

04

RECORD

external id

공식 문서를 구경하지 않고, 코드 실행과 웹 결과를 연결합니다

LCEL 실행 → StateGraph 조립 → interrupt·resume → Vercel 결과 저장

도구·서비스	도메인·역할	이번 강의에서 쓰는 이유	유사 도구와의 차이
LangChain	모델·Prompt·Tool 조합	개별 AI 구성요소를 공통 인터페이스로 연결	단순 호출은 SDK만으로도 가능
LangGraph	상태 기반 Workflow	분기·중단·승인·재개를 State로 명시	짧은 직선 흐름에는 과할 수 있음
LangSmith	관측·평가	trace·dataset·experiment를 같은 실행 계보로 확인	Local JSONL은 무료지만 UI·협업이 약함
Pydantic·pytest	계약·결정론적 검사	Schema와 실패 조건을 빠르고 재현 가능하게 검증	의미 품질 평가는 별도 rubric 필요

실제 실행 · LCEL 네 단계와 Policy Check

이 화면에서 딱 볼 것

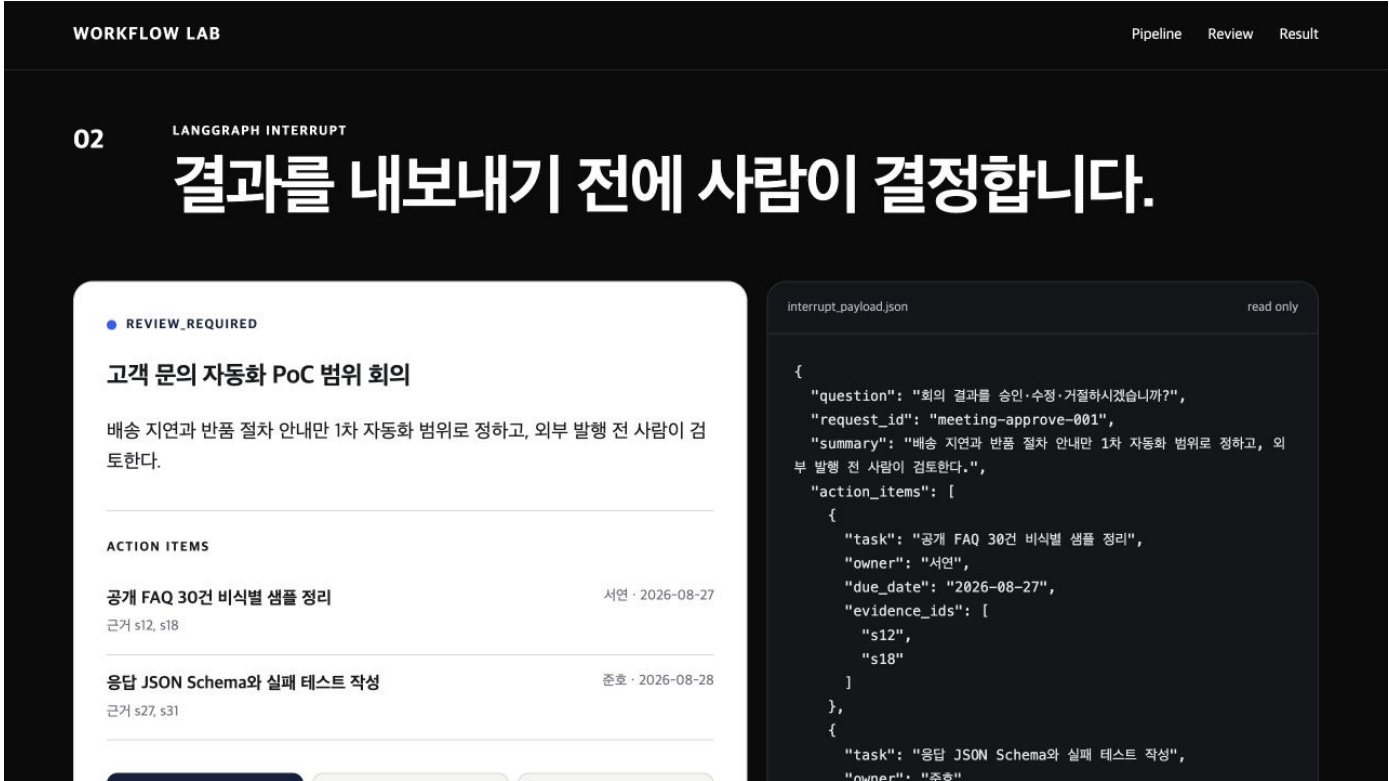
- Prompt→Adapter→Parser→Validator
- typed output
- 네 정책 검사 통과
- Python 결과 JSON을 UI에서 읽음

The screenshot displays the 'WORKFLOW LAB' interface. At the top, there are tabs for 'Pipeline', 'Review', and 'Result'. The main content area is titled '01 LANGCHAIN LCEL 자연어를 업무 데이터로 바꾸는 네 단계'. Below this title, there are four numbered steps in boxes: 01 ChatPromptTemplate, 02 Model Adapter, 03 PydanticOutputParser, and 04 Policy Validator. Below the steps, there are two main sections: 'INPUT' and 'POLICY CHECK'. The 'INPUT' section shows the text '한국어 합성 회의' and a file path 'data/meeting_sample_ko_12min.txt', with a note that the original text is replaced by the generated result. The 'POLICY CHECK' section shows a dark blue box with the title '모든 정책 검사 통과' and a list of four passed checks: 'PASS · schema_valid', 'PASS · evidence_present', 'PASS · automatic_email_blocked', and 'PASS · human_approval_required'. At the bottom right, there is a caption: 'Vercel Demo의 실제 LangChain 결과 화면'.

실제 실행 · interrupt payload에서 멈춤

이 화면에서 딱 볼 것

- REVIEW_REQUIRED
- 초안·Action Item·근거
- approve·edit·reject
- 자동 메일 false



WORKFLOW LAB

Pipeline Review Result

02 LANGGRAPH INTERRUPT

결과를 내보내기 전에 사람이 결정합니다.

● REVIEW_REQUIRED

고객 문의 자동화 PoC 범위 회의

배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외부 발행 전 사람이 검토한다.

ACTION ITEMS

공개 FAQ 30건 비식별 샘플 정리	서연 · 2026-08-27
근거 s12, s18	
응답 JSON Schema와 실패 테스트 작성	준호 · 2026-08-28
근거 s27, s31	

interrupt_payload.json read only

```
{
  "question": "회의 결과를 승인·수정·거절하시겠습니까?",
  "request_id": "meeting-approve-001",
  "summary": "배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외부 발행 전 사람이 검토한다.",
  "action_items": [
    {
      "task": "공개 FAQ 30건 비식별 샘플 정리",
      "owner": "서연",
      "due_date": "2026-08-27",
      "evidence_ids": [
        "s12",
        "s18"
      ]
    },
    {
      "task": "응답 JSON Schema와 실패 테스트 작성",
      "owner": "준호"
```

Vercel Demo의 실제 사람 검토 화면

실제 실행 · 거절은 REJECTED와 HOLD

이 화면에서 딱 볼 것

- 같은 thread 재개
- terminal state
- release gate
- local JSON only

WORKFLOW LAB

Pipeline Review Result

03

COMMAND RESUME

같은 thread의 최종 상태를 확인합니다.

TERMINAL STATUS

REJECTED

외부로 내보내지 않고 검토 상태로 종료합니다.

RELEASE GATE

HOLD

Schema·근거·사람 결정·외부 쓰기 금지를 검사합니다.

SIDE EFFECT

LOCAL JSON ONLY

메일·결제·고객 데이터 변경을 수행하지 않습니다.

AUDIT EVENTS

1. **validate**
REVIEW_REQUIRED

2. **review**
reject

3. **rejected**

OUTPUT

내 결정이 담긴 JSON

브라우저 Demo 결과를 파일로 저장해 Git commit 또는 포트폴리오 증거로 사용할 수 있습니다.

Vercel Demo의 실제 거절 결과

실제 실행 · Python 3.12 test 25개 통과

이 화면에서 딱 볼 것

- LangChain typed output
- interrupt·resume
- 승인·수정·거절
- trace·READY/HOLD

5차시 · 점심 전 전체 회귀 test

새로 만든 Ollama 경로뿐 아니라 기존 Agent·LangChain·LangGraph·STT 계약이 함께 유지되는지 전체 Day 1 test를 실행합니다. 실패하면 마지막 줄보다 첫 번째 오류부터 복구합니다.

```
In [9]: full_test = subprocess.run(
        [sys.executable, "-m", "pytest", "-q"],
        cwd=ROOT,
        capture_output=True,
        text=True,
        check=False,
    )
    print(full_test.stdout.strip())
    assert full_test.returncode == 0, full_test.stderr

.....
25 passed in 0.33s
```

6차시 · LangChain LCEL Provider 교체

같은 transcript와 Pydantic schema를 유지하고 provider만 바꿉니다.

Prompt → fixture/Ollama → Pydantic Parser → Policy Validator

Ollama용 선택 패키지는 다음과 같이 설치합니다.

python -m pip install -r requirements-local-llm-optional.txt

```
In [10]: from src.langchain_lab import run_langchain_lab

transcript = (ROOT / "data/meeting_sample_ko_12min.txt").read_text(encoding="utf-8")
fixture_chain = run_langchain_lab(transcript, provider="fixture")
print(json.dumps({
    "provider_used": fixture_chain["provider_used"],
    "pipeline": fixture_chain["pipeline"],
    "checks": fixture_chain["checks"],
}, ensure_ascii=False, indent=2))

{
  "provider_used": "fixture",
  "pipeline": [
    "fixture_chain"
  ]
}
```

04 notebook의 실제 전체 test 결과

실제 WorkflowState

코드 전체보다 경계 한 줄을 먼저 봅니다

- 다음 node에 필요한 필드
- 승인 event 유지
- 오류 누적
- 외부 쓰기 정책

```
class WorkflowState(TypedDict, total=False):  
    request_id: str  
    draft: dict[str, Any]  
    validation_errors: list[str]  
    status: str  
    review: dict[str, Any]  
    audit_events: list[dict[str, Any]]  
    export_ready: bool  
    automatic_email: bool
```

실제 조건 route

코드 전체보다 경계 한 줄을 먼저 봅니다

- 오류 필드 기반
- review 또는 failed
- 암묵적 분기 없음
- test로 두 경로 검증

```
def route_after_validate(state):  
    return (  
        "review"  
        if not state.get("validation_errors")  
        else "failed"  
    )
```


사람 검토에 넘기는 데이터 구조

코드 전체보다 경계 한 줄을 먼저 봅니다

- 초안·근거 한 화면
- 직렬화 가능한 값
- 자동 메일 금지
- 결정은 resume 값

```
human_input = interrupt({  
    "request_id": state["request_id"],  
    "summary": state["draft"]["summary"],  
    "action_items": state["draft"]["action_items"],  
    "options": ["approve", "edit", "reject"],  
    "automatic_email": False  
})
```

Command로 같은 thread 재개

코드 전체보다 경계 한 줄을 먼저 봅니다

- thread_id 유지
- checkpoint load
- resume value
- new thread와 구분

```
config = {"configurable": {"thread_id": request_id}}  
first = graph.invoke(input_state, config)  
# interrupt에서 멈춤  
second = graph.invoke(Command(resume={"decision": "approve"}), config)
```

승인 event

코드 전체보다 경계 한 줄을 먼저 봅니다

- 누가
- 무슨 결정
- 이유
- 수정 payload·시각

```
approval = {  
  "decision": "edit",  
  "reviewer": "role-reviewer",  
  "reason": "담당자 근거 부족",  
  "edited_payload": corrected,  
  "at": iso_time  
}
```

소프트웨어 실습 1 · LangChain LCEL 실행

07 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 7차시 notebook을 연다.
- 2 Prompt→Adapter→Parser→Validator 셀을 실행한다.
- 3 MeetingBrief 타입과 policy check를 확인한다.
- 4 Prompt 한 줄을 바꾸고 다시 실행한다.

완료 증거 | typed JSON + 네 policy check

소프트웨어 실습 2 · StateGraph 조립

07 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 build_graph 코드를 연다.
- 2 validate·review·finalize node를 확인한다.
- 3 조건 edge와 END를 연결한다.
- 4 InMemorySaver로 graph를 compile한다.

완료 증거 | 실행 가능한 StateGraph

소프트웨어 실습 3 · interrupt와 같은 thread 재개

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 최초 invoke를 실행한다.
- 2 __interrupt__ payload를 출력한다.
- 3 같은 thread_id로 Command(resume)를 실행한다.
- 4 READY_FOR_EXPORT state를 저장한다.

완료 증거 | interrupt payload + 승인 state

소프트웨어 실습 4 · 세 결정과 웹 결과 저장

10 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 approve·edit·reject를 각각 실행한다.
- 2 READY·HOLD terminal state를 비교한다.
- 3 Vercel Demo에서 같은 결정을 눌러본다.
- 4 내 결정이 담긴 JSON을 내려받는다.

완료 증거 | 상태 3개 + 결과 JSON

실패 1 · State에 모든 것을 저장

현업에서 자주 깨지는 방식

- 거대한 message history
- binary audio 포함
- 개인정보 장기 보존
- node coupling

제가 먼저 고치는 순서

- 최소 structured state
- 원본은 보호 저장소
- reference ID
- node별 계약

실패 2 · interrupt를 try/except로 감쌘

현업에서 자주 깨지는 방식

- 중단 신호를 오류 처리
- 재개 불가
- state 혼란
- 반복 호출

제가 먼저 고치는 순서

- 공식 interrupt 규칙
- 직렬화 payload
- 같은 thread
- 명시적 resume

실패 3 · 승인 전에 외부 쓰기

현업에서 자주 깨지는 방식

- 재실행 중복
- 오류 확산
- 사람이 취소 불가
- 감사 추적 부재

제가 먼저 고치는 순서

- draft·preview
- interrupt
- approved payload
- idempotent publish

재직자 · Human Approval 운영표

승인 기준

- 개인정보 flag
- 낮은 confidence
- 외부 발행
- 금액·계약 영향

운영 증거

- reviewer role
- SLA
- decision reason
- audit log

Gate 7 · 승인 뒤 같은 요청이 다시 왔습니다

상황: reviewer가 승인한 직후 네트워크가 끊겼고, 같은 request_id로 publish가 다시 호출됐습니다.

1

어떤 State와 checkpoint에서 재개할까?

2

승인 event에서 어떤 근거와 결정을 읽을까?

3

새로 발행할까, 저장된 성공 결과를 돌려줄까?

16:20-17:00

1일차 8차시

STT와 LangSmith

16:20-16:28 강의 · 16:28-16:36 실행 시연 · 16:36-16:53 소프트웨어 실습 · 16:53-17:00 판단

지금 배우는 내용	이 차시에서 이해할 것	함께 챙길 내용
STT 품질 · Trace · Dataset · 평가	실행 기록으로 READY와 HOLD를 결정한다	STT 품질 기준 + READY/HOLD 결정

관측은 READY와 HOLD를 가를 때 끝납니다

다음 버전을 배포하지 말아야 할 근거를 trace와 dataset에서 바로 찾을 수 있는가?

입력

Managed 또는 Local STT의 segment를 받음

품질

무음·반복·이름·날짜 오류를 flag

계보

node·LLM·tool을 trace tree로 연결

비교

같은 dataset으로 baseline과 candidate 실행

결정

품질·안전·지연 기준으로 READY/HOLD

관측과 배포 판단을 잇는 네 단어

용어	쉽게 말하면	오늘 쓰는 곳
STT Speech to Text	사람의 음성을 문자와 타임스탬프로 변환하는 기술이다.	faster-whisper가 audio를 segment 배열로 변환한다.
Trace Execution Record	한 요청 안에서 Node·LLM·Tool이 어떤 입력과 출력을 냈는지 연결한 실행 기록이다.	LangSmith run tree에서 Graph 아래 child run을 확인한다.
Dataset / Experiment Fixed Inputs / Compared Runs	Dataset은 평가 입력 묶음이고 Experiment는 같은 입력으로 특정 버전을 실행한 결과 묶음이다.	day2-v1 데이터셋에 baseline과 candidate를 각각 실행한다.
Release Gate Ship or Hold Rule	품질·안전·지연시간 기준을 모두 통과할 때만 배포를 허용하는 규칙이다.	Schema 98%·근거 연결 90%·오발행 0건을 모두 만족해야 READY이다.

Managed 회의 기능은 현업의 현재 출발점

Google Meet 등은 전사·노트 기능을 제공하지만 계정·edition·관리자 설정에 의존한다.

- 빠른 활용
- 정책·보존
- 지원 언어·edition 확인

배포 전에 보는 신호 | STT가 틀리면 LLM은 틀린 입력을 더 그럴듯하게 정리한다.

수업의 local STT는 원리를 이해하는 최소 실습

faster-whisper 또는 whisper.cpp로 audio→segment contract를 직접 확인한다.

- offline 가능
- timestamp
- model·device 선택

STT 품질은 WER 하나로 끝나지 않는다

업무에서는 이름·숫자·날짜·Action Item 근거 오류를 별도로 본다

•

- empty·repetition
- entity error
- timestamp
- human correction

배포 전에 보는 신호 | Trace는 과거 기록이 아니라 다음 실험 질문을 만드는 재료다.

LangSmith trace는 실행의 계보

graph run 아래 node·LLM·tool의 input/output·latency·error를 연결한다.

- parent-child
- metadata
- tags
- error

민감 데이터는 trace 전에 줄인다

교육 계정에는 공개·합성 샘플만 전송하고 원문·secret을 masking한다.

- anonymize
- sampling
- local JSONL
- retention 확인

Dataset이 평가 입력을 고정

golden example과 실패 사례를 dataset으로 만들고 experiment를 반복한다.

- input
- reference output
- metadata
- version

배포 전에 보는 신호 | 평가는 점수를 내는 일이 아니라 READY와 HOLD를 가르는 운영 결정이다.

전체 pipeline

01

AUDIO

metadata·STT

02

DRAFT

LLM·schema

03

REVIEW

evidence·interrupt

04

PUBLISH

승인·idempotency

STT 실행 순서

01

CHECK

format·duration

02

TRANSCRIBE

segment

03

QUALITY

empty·repeat

04

FALLBACK

정답 transcript

Trace 설계 순서

01

QUESTION

무엇을 알고 싶나

02

FIELDS

어떤 metadata

03

BOUNDARY

무엇을 숨기나

04

VIEW

run tree·filter

Dataset 구축

01

SEED

golden 3-5건

02

FAIL

예외 추가

03

VERSION

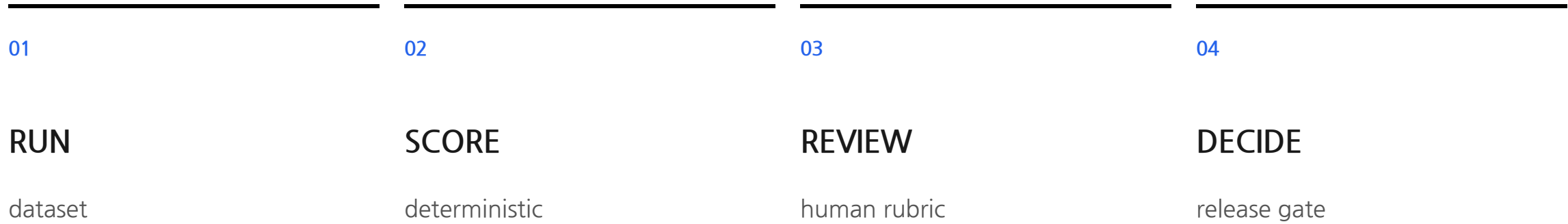
변경 기록

04

EXPERIMENT

같은 입력 비교

평가 루프



STT와 LangSmith 화면은 하나의 품질 루프로 연결됩니다

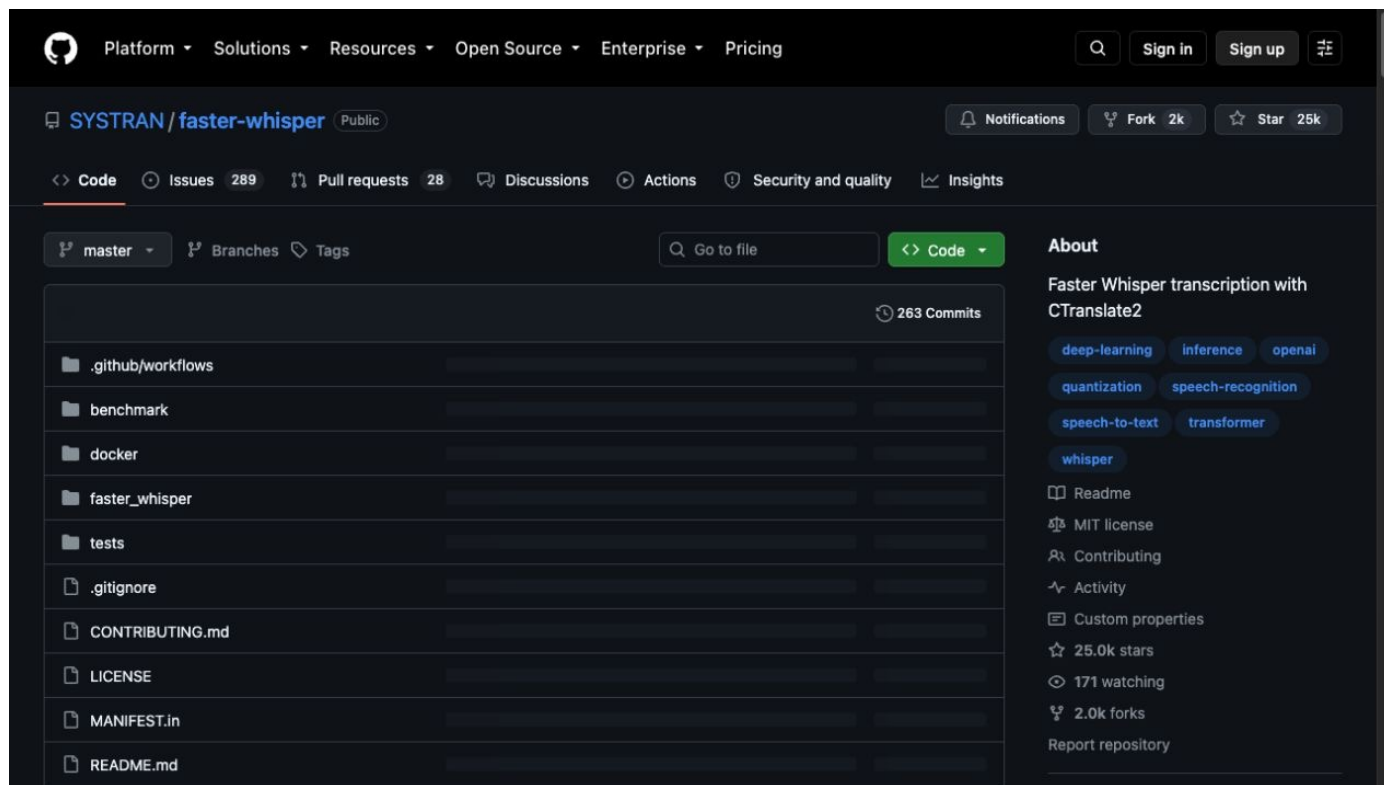
Local STT 두 경로 → Managed 노트 → trace tree → experiment

도구·서비스	도메인·역할	이번 강의에서 쓰는 이유	유사 도구와의 차이
Google Meet	Managed 회의 STT	이미 제공되는 전사·노트를 현업 출발점으로 사용	Zoom·Teams도 유사하지만 구독 정책이 다름
faster-whisper	Python Local STT	segment·timestamp를 코드에서 직접 다루기 쉬움	whisper.cpp보다 Python 통합이 편함
whisper.cpp	경량 CLI STT	낮은 사양·다양한 환경의 local 대안	Python pipeline 연결은 추가 작업
LangSmith Trace	실행 관측	node·LLM·tool의 지연·오류 계보를 한 화면에서 확인	민감 데이터는 masking 또는 local trace
LangSmith Eval	Dataset·Experiment	같은 입력으로 baseline과 candidate를 비교	pytest는 구조 검사에 강하고 의미 평가는 제한

faster-whisper · Python 기반 local STT

이 화면에서 딱 볼 것

- Whisper 재구현
- model·device·compute_type
- segments generator
- 수업 전 model 준비

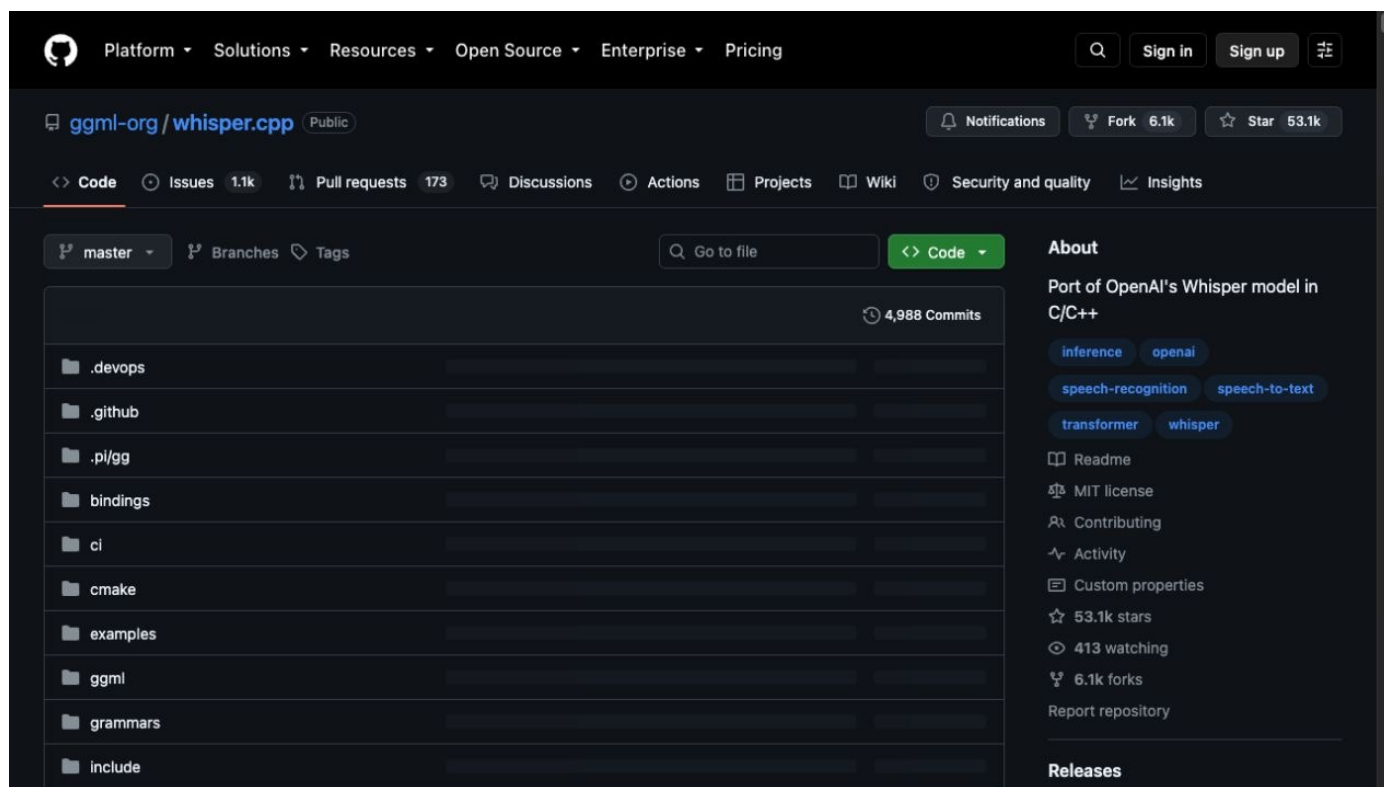


faster-whisper 공식 GitHub

whisper.cpp · 경량 local 대안

이 화면에서 딱 볼 것

- C/C++ 기반
- 다양한 환경
- CLI 실습 대안
- 모델 파일 사전 준비



whisper.cpp 공식 GitHub

Google Meet 자동 노트 · 만들기보다 통합하기

이 화면에서 딱 볼 것

- 구독·관리자 설정 의존
- 기존 전사·요약 활용
- local은 예외·통제 학습
- 지원 범위 강의 전 확인

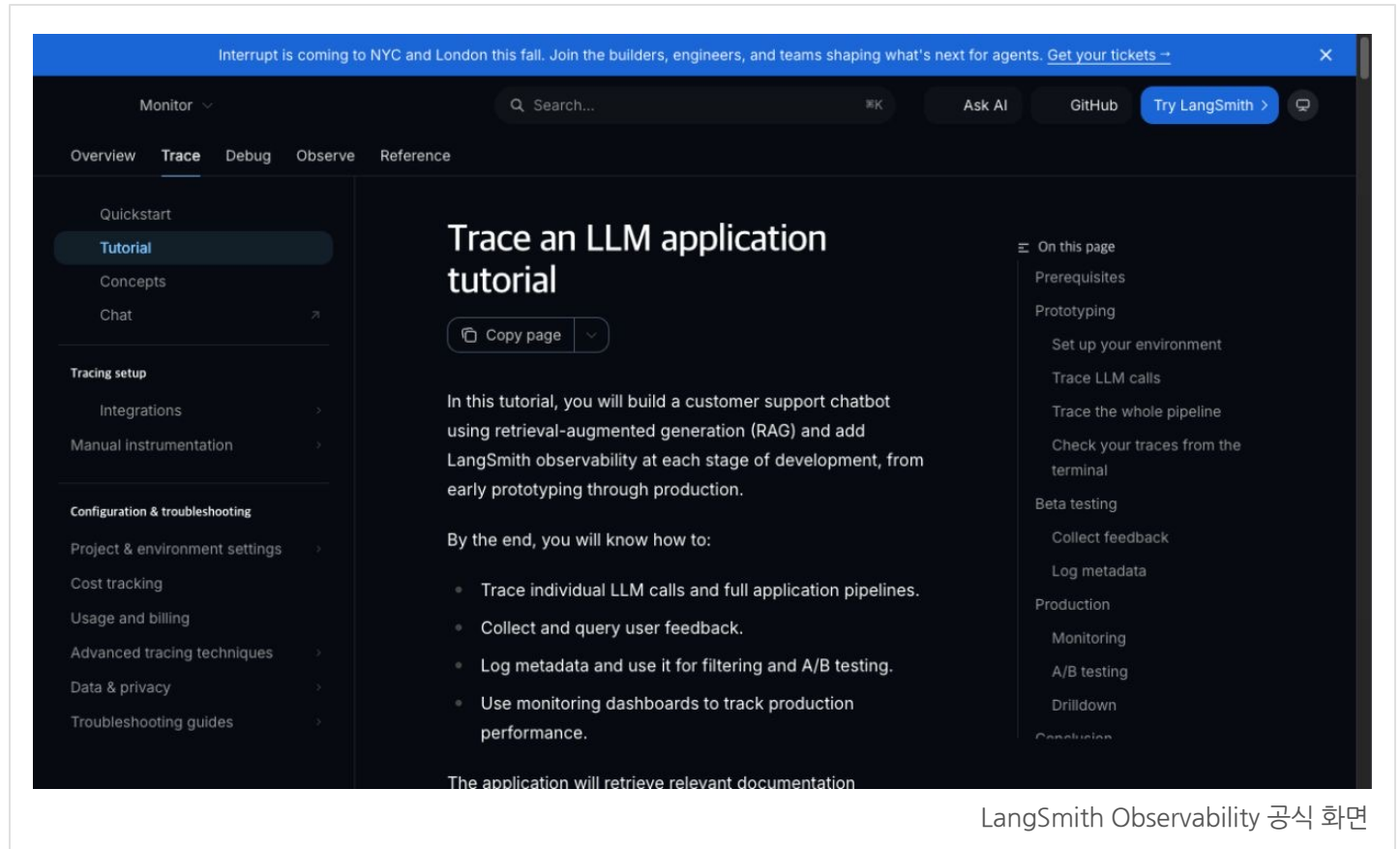
The screenshot shows the Google Meet Help Center interface. At the top, there's a search bar and a navigation menu. The main content area is titled 'Google Meet의 자동 회의록' (Google Meet's automatic meeting notes). Below the title, it says '중요:' (Important:). The content lists two main points: 1. To use this feature, you need to meet the requirements of Google Workspace version or Google AI add-on subscription. 2. This feature can be used in various languages. A list of supported languages is provided: English, French, German, Italian, Japanese, Korean, Portuguese, and Spanish. Below this, a note states that the feature only supports one language at a time and that using multiple languages in a single meeting is not supported. A link is provided to learn more about changing the meeting language. On the right side, there's a 'Help Center' sidebar with several links: 'Gemini 기반 Google Workspace 시작하기', 'Google Meet의 Gemini로 배경 이미지 만들기', 'Enhance your video & audio with Gemini in Google Meet', 'Google Meet의 자동 회의록', '오프라인 회의에 자동 회의록 사용하기', 'Meet에서 스마트 오디오 사용하기', 'Google Meet의 Gemini에게 물어보기', '통역 기능에 대해 알아보기', 'Calendar 및 Meet의 자동 회의록 설정', and 'Workspace의 Gemini 대화 기록 찾기 및 관'.

Google Meet Take notes for me 도움말

LangSmith observability · trace tree

이 화면에서 딱 볼 것

- node별 input/output
- latency·token·error
- metadata·tag
- 공개 샘플만

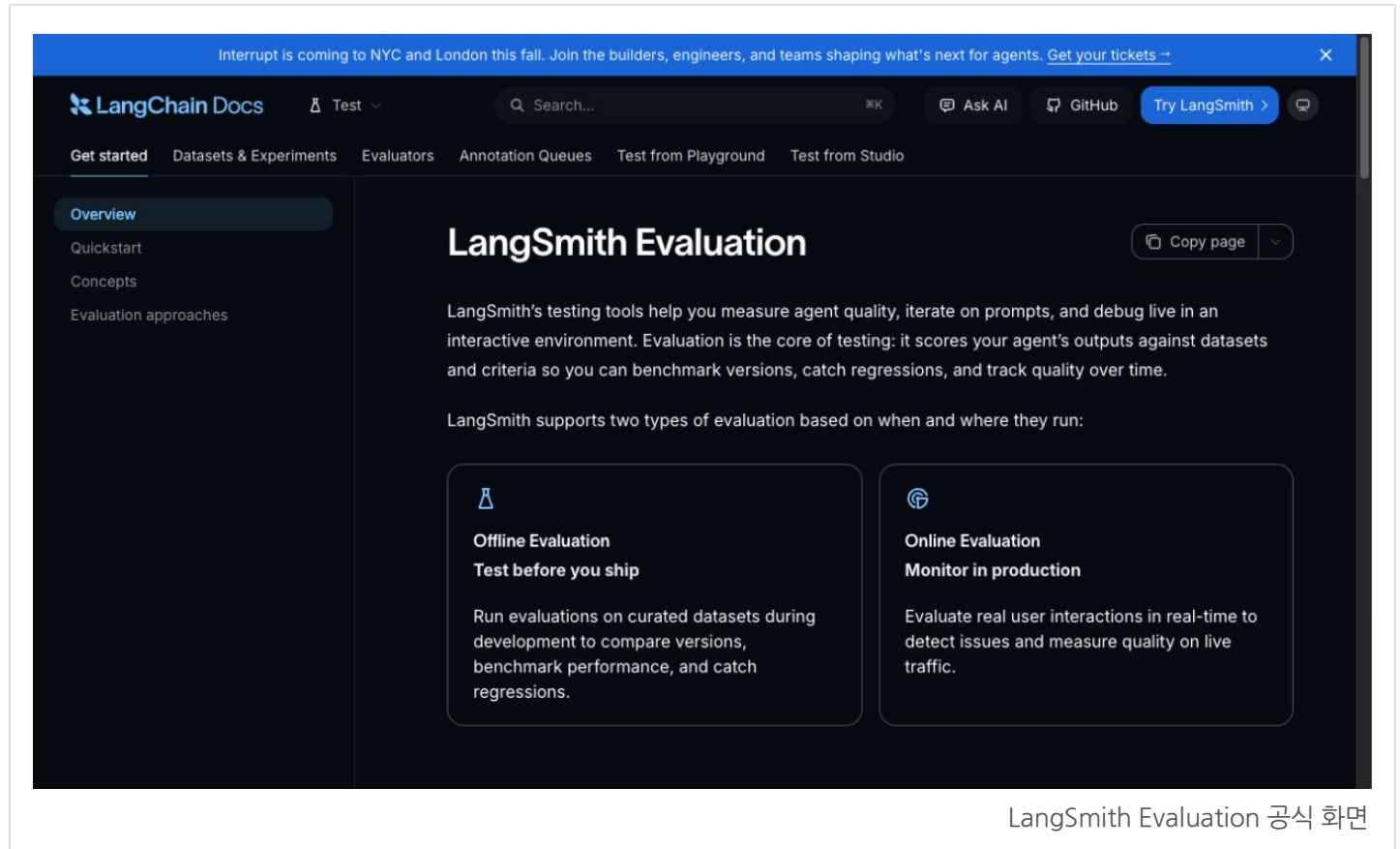


LangSmith Observability 공식 화면

LangSmith evaluation · dataset과 experiment

이 화면에서 딱 볼 것

- offline evaluation
- online monitoring
- human feedback
- baseline 비교



faster-whisper 최소 호출

코드 전체보다 경계 한 줄을 먼저 봅니다

- CPU int8 경로
- language 명시
- segment timestamp
- generator 소비

```
from faster_whisper import WhisperModel
model = WhisperModel("small", device="cpu", compute_type="int8")
segments, info = model.transcribe(audio_path, language="ko")
rows = [{"start": s.start, "end": s.end, "text": s.text}
         for s in segments]
```

STT 품질 flag

코드 전체보다 경계 한 줄을 먼저 봅니다

- 무음
- 반복
- 지나치게 희소
- 사람 검토

```
flags = []  
if not text.strip(): flags.append("EMPTY")  
if repeated_ratio(text) > 0.35: flags.append("REPETITION")  
if duration > 0 and len(text) / duration < 0.5: flags.append("TOO_SPARSE")  
if flags: status = "NEEDS_REVIEW"
```

LangSmith 환경 변수 경계

코드 전체보다 경계 한 줄을 먼저 봅니다

- 프로젝트 분리
- key 환경 변수
- 교육 샘플만
- tracing off fallback

```
LANGSMITH_TRACING=true  
LANGSMITH_PROJECT=ipa-k-work-copilot  
# LANGSMITH_API_KEY is set outside notebook  
# never print or commit the key
```

Trace metadata

코드 전체보다 경계 한 줄을 먼저 봅니다

- dataset version
- provider·model
- thread 연결
- 민감성·fallback

```
metadata = {  
    "dataset_version": "day2-v1",  
    "provider": "ollama",  
    "model": "qwen3:4b",  
    "thread_id": request_id,  
    "contains_pii": False,  
    "fallback_used": False  
}
```

결정론적 evaluator

코드 전체보다 경계 한 줄을 먼저 봅니다

- 근거 연결률
- schema pass
- 계산 재현
- release threshold

```
def evidence_rate(output):
    claims = output["decisions"] + output["action_items"]
    linked = [c for c in claims if c.get("evidence_ids")]
    return len(linked) / max(1, len(claims))

def schema_pass(output):
    return MeetingResult.model_validate(output) is not None
```

Ideation 1 · Managed vs Local 선택

05 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 현재 조직의 회의 도구를 적는다.
- 2 전사·요약 제공 여부를 적는다.
- 3 계정·보존·권한 제약을 적는다.
- 4 직접 STT가 필요한 이유를 한 줄로 정한다.

완료 증거 | Managed/Local 선택 한 줄

소프트웨어 실습 1 · STT quality gate 실행

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 meeting_demo.py를 fixture mode로 실행한다.
- 2 segment의 quality_flags를 확인한다.
- 3 READY/HOLD와 fallback_reason을 확인한다.
- 4 transcript.json을 저장한다.

완료 증거 | quality gate JSON

소프트웨어 실습 2 · 로컬 trace 생성

08 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 workflow_service를 approve로 실행한다.
- 2 trace.json의 세 span을 연다.
- 3 latency·status·metadata를 확인한다.
- 4 reject로 다시 실행해 차이를 본다.

완료 증거 | approve/reject trace 2개

소프트웨어 실습 3 · Release Gate 테스트

09 MIN

강사 화면을 보며
각자 진행합니다

막히면 채팅창에
첫 오류 한 줄만
남겨주세요

- 1 pytest에서 READY test를 실행한다.
- 2 reject가 HOLD인지 확인한다.
- 3 evidence_rate와 자동 메일 금지를 확인한다.
- 4 LangSmith 연결 전 local fallback 상태를 저장한다.

완료 증거 | READY/HOLD test + trace

실패 1 · STT 결과를 정답으로 간주

현업에서 자주 깨지는 방식

- 이름·숫자 오류
- 무음 반복
- 화자 혼동
- timestamp 유실

제가 먼저 고치는 순서

- quality flags
- 원문 audio link
- 사람 correction
- segment evidence

실패 2 · 모든 trace에 원문 전송

현업에서 자주 깨지는 방식

- 개인정보
- 회사 기밀
- 보존 정책 불명
- 교육 계정 혼용

제가 먼저 고치는 순서

- 공개·합성 샘플
- masking
- metadata 최소화
- local JSONL fallback

실패 3 · trace만 보고 평가 완료

현업에서 자주 깨지는 방식

- 관찰은 가능
- 정답 기준 없음
- 회귀 판단 불가
- release 결정 없음

제가 먼저 고치는 순서

- dataset
- evaluator
- baseline
- gate·HOLD

재직자 · 운영 dashboard 질문

매일 볼 것

- failure rate
- review queue
- latency
- cost·fallback

배포 전 볼 것

- dataset score
- schema·evidence
- false publish
- rollback 기준

17:30-18:00 · Q&A와 Exit Ticket

오늘 막힌 부분을 먼저 복구하고, 마지막에는 내 Workflow를 READY 또는 HOLD로 판단합니다.

1

내 흐름에서 자동 실행과 사람 승인의 경계는 어디인가?

2

실패 뒤 다시 시작할 checkpoint와 fallback은 무엇인가?

3

어떤 trace·dataset 신호가 나오면 다음 배포를 HOLD할 것인가?